VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



SOFTWARE REQUIREMENT SPECIFICATION

SEMESTER 252 ACADEMIC YEAR 2025-2026

MUSEUM COMPANION APPLICATION

COURSE: MOBILE APPLICATION DEVELOPMENT (CO3041)

ADVISOR: PhD. Phan Trần Minh Khuê

GROUP: CC01 - 03

STUDENT 1: Nguyễn Tiến Khang – 2252304

STUDENT 2: Trần Nguyễn Anh Khoa – 2252364

STUDENT 3: Ôn Đức Phát– 2352897

STUDENT 4: Phan Quang Minh – 2252486

HO CHI MINH CITY, MAY 2026

Member Contributions

Name	Student ID	Main Contributions	Completion
Nguyen Tien Khang	2252304	System architecture, frontend and backend implementation, RAG assistant integration, database design, UI-UX design, testing, and documentation.	**150**%
On Duc Phat	2352897	Frontend development, API integration, QR-code features, testing, and documentation.	100%
Tran Nguyen Anh Khoa	2252364	Backend development, database integration, authentication features, testing, and deployment support.	100%
Phan Quang Minh	2252486	Market research, UI-UX prototyping, testing, report formatting, and presentation preparation.	100%

Contents

Member Contributions **i**

1 Introduction **1**

1.1 System Overview 1

1.1.1 Artwork Information and Multimedia Interpretation 2

1.1.2 Search and Intelligent Exploration 3

1.1.3 Curated and Specialized Tours 3

1.1.4 Collections and Personalized Content Organization 4

1.1.5 Interactive Map and Location-Aware Navigation 4

1.1.6 User Accounts and Personalization 5

1.1.7 Summary 5

2 Requirement Analysis **6**

2.1 Stakeholders 6

2.1.1 Primary Stakeholders 6

2.1.2 Secondary Stakeholders 7

2.2 Functional Requirements 7

2.2.1 Artwork Information and Access 7

2.2.2 Search, QR Code, and Discovery 8

2.2.3 Intelligent Assistant 9

2.2.4 Tours and Collections 9

2.2.5 Personalization and Saved Content 9

2.2.6 Interactive Map and Location Features 10

2.2.7 Content Management 10

2.2.8 System and Data Synchronization 11
2.3 Non-Functional Requirements 11
2.3.1 Performance 11
2.3.2 Usability 11
2.3.3 Reliability 12
2.3.4 Security 12
2.3.5 Scalability 12
2.3.6 Maintainability 12
2.3.7 Compatibility 13
2.3.8 Accessibility 13
2.4 Use Cases 13
2.4.1 Use Case List 15
2.4.2 Detailed Use Case Tables 16
2.5 Context Diagram 29
2.6 Data Flow Diagram 31
2.6.1 Interpretation of the Data Flow Diagram 34
2.6.2 Summary 35
2.7 Market Research 35
2.7.1 Target Audience Analysis 36
2.7.2 Identified Problems in Current Museum Experiences 36
2.7.3 Case Study: iMuseum VFA 37
2.7.4 Case Study: Bảo Tàng Lịch Sử TP.HCM 38
2.7.5 Case Study: Smartify 39
2.7.6 Design Implications for the Proposed System 41
2.7.7 Competitive Positioning 42
2.7.8 Summary 42

3 UI-UX Design **43**
3.1 Introduction 43
3.2 Design Objectives 44
3.3 Target User Considerations 45
3.3.1 Students 45

3.3.2 Young Adults . . . 45
3.3.3 Adults and Senior Visitors . . . 45
3.3.4 Disadvantaged Users and Accessibility Needs . . . 46
3.4 Design Philosophy . . . 46
3.4.1 Usability-Oriented Design . . . 47
3.4.2 Inclusive Design Principles . . . 47
3.4.3 Museum-Centered Visual Design . . . 48
3.5 Color System . . . 49
3.5.1 Background and Neutral Colors . . . 49
3.5.2 Primary Text and Contrast . . . 49
3.5.3 Accent Colors . . . 49
3.5.4 Dark Mode . . . 50
3.6 Typography . . . 50
3.6.1 Playfair Display . . . 50
3.6.2 Inter . . . 51
3.6.3 Typography Hierarchy . . . 51
3.7 Information Architecture . . . 52
3.8 Navigation and Interaction Design . . . 52
3.9 QR-Code Interaction Design . . . 53
3.10 Interactive Map Design . . . 53
3.11 Accessibility and Inclusive Design . . . 54
3.12 Mockup Interfaces . . . 55
3.13 Summary . . . 58

4 System Design **59**
4.1 Sequence Diagrams . . . 59
4.1.1 Artwork Search and Retrieval Flow . . . 60
4.1.2 Authentication and QR-Code Interaction Flow . . . 62
4.1.3 Architectural Interpretation . . . 64
4.1.4 Summary . . . 64
4.2 Relational Schema . . . 65
4.2.1 Core Museum Entities . . . 66

4.2.2 Tour and Exploration Entities . . . 69
4.2.3 User and Personalization Entities . . . 70
4.2.4 Assistant and Conversational Entities . . . 72
4.2.5 Relationship Analysis . . . 73
4.2.6 Schema Design Considerations . . . 73
4.2.7 Summary . . . 75
4.3 Class Diagram . . . 75
4.3.1 Controller Layer . . . 77
4.3.2 Authentication and Guard Layer . . . 78
4.3.3 Service Layer . . . 79
4.3.4 Infrastructure and Adapter Layer . . . 82
4.3.5 DTOs and Response Models . . . 83
4.3.6 Domain Entity Layer . . . 84
4.3.7 Architectural Interpretation . . . 88

5 System Architecture 89
5.1 Architectural Overview . . . 89
5.2 High-Level System Components . . . 90
5.2.1 iOS Mobile Application . . . 90
5.2.2 Backend API Server . . . 91
5.2.3 PostgreSQL Database . . . 92
5.2.4 Vector Database . . . 92
5.2.5 Firebase Authentication . . . 93
5.2.6 Gemini 2.5 Flash Integration . . . 93
5.3 Detailed Backend Architecture . . . 93
5.3.1 Museum Module . . . 94
5.3.2 Tour Module . . . 94
5.3.3 Assistant Module . . . 95
5.3.4 RAG Module . . . 95
5.3.5 Recommendation Module . . . 96
5.3.6 Firebase Authentication Layer . . . 96
5.4 Frontend Architecture Using MVVM . . . 96

5.4.1 View Layer . . . 96
5.4.2 ViewModel Layer . . . 97
5.4.3 Model Layer . . . 97
5.4.4 Advantages of MVVM . . . 98
5.5 Backend Architecture Using Modular Monolith . . . 98
5.5.1 Reasons for Choosing a Modular Monolith . . . 98
5.5.2 Module Isolation . . . 99
5.5.3 Advantages of the Modular Monolith Approach . . . 100
5.6 Architectural Characteristics . . . 100
5.6.1 Layered Separation of Concerns . . . 100
5.6.2 AI Isolation . . . 101
5.6.3 Hybrid Recommendation Architecture . . . 101
5.6.4 Offline Support . . . 101
5.6.5 Scalability and Extensibility . . . 101
5.7 Summary . . . 102

6 Deployment **103**
6.1 Technology Stack . . . 103
6.1.1 Frontend Technologies . . . 103
6.1.2 Backend Technologies . . . 103
6.1.3 Database Technologies . . . 104
6.1.4 Authentication and Cloud Services . . . 104
6.1.5 Artificial Intelligence Technologies . . . 104
6.1.6 Development and Deployment Tools . . . 105
6.2 Deployment . . . 105
6.3 Testing Plans . . . 106
6.3.1 Functional Testing . . . 106
6.3.2 Frontend Testing . . . 107
6.3.3 Backend and API Testing . . . 107
6.3.4 Database Testing . . . 108
6.3.5 Assistant and AI Testing . . . 108
6.3.6 Performance Testing . . . 109

6.3.7 Usability Testing 109
6.3.8 Integration Testing 110
6.3.9 Summary 110

7 Conclusion **111**

List of Figures

2.1 Use-case diagram of the Museum Companion System 14
2.2 Context diagram of the Museum Companion System 31
2.3 Level 0 Data Flow Diagram of the Museum Companion System 34
2.4 iMuseum VFA's basic UI 37
2.5 iMuseum VFA's basic operations 38
2.6 Basic operations of Bảo Tàng Lịch Sử TPHCM 39
2.7 Smartify UI 40
2.8 Smartify UI is considered more minimalistic but feature-rich 41

3.1 Colors for aesthestics and usability 47
3.2 Nielsen's Heuristics 48
3.3 Login 55
3.4 Search 55
3.5 Tour and Artwork 56
3.6 QR Code 56
3.7 Dark Mode 57
3.8 Vietnamese 57
3.9 Table UI 58

4.1 Sequence Diagrams 59
4.2 Relational schema of the Museum Companion System 66
4.3 Class diagram of the Museum Companion System 76

5.1 High-level architecture of the Museum Companion System 90
5.2 Detailed backend and frontend architecture 94

Chapter 1

Introduction

1.1 System Overview

The proposed system is designed as an intelligent *Museum Companion System* that functions as a virtual guide and interactive assistant for visitors within an art museum environment. The system aims to enhance traditional museum experiences by combining multimedia interpretation, personalized exploration, spatial navigation, and intelligent contextual assistance into a unified mobile application.

Unlike conventional museum guide applications that primarily provide static textual information, the proposed system emphasizes immersive and user-centered interaction. The application accompanies visitors throughout their museum journey, allowing them to explore artworks, collections, and exhibitions through multiple forms of interaction, including direct browsing, intelligent searching, QR-code scanning, and guided navigation.

At its core, the system bridges the gap between physical exhibits and digital interpretation. By integrating artwork information, multimedia content, location-aware services, and intelligent retrieval mechanisms, the system enables visitors to engage with cultural heritage in a more meaningful, accessible, and interactive manner.

1.1.1 Artwork Information and Multimedia Interpretation

One of the primary functionalities of the system is the provision of comprehensive digital representations for artworks displayed within the museum. Each artwork entry contains detailed metadata and interpretive content designed to support both casual visitors and users seeking deeper artistic or historical understanding.

For every artwork, the system provides textual descriptions explaining the historical background, artistic significance, cultural context, and technical characteristics of the piece. These descriptions are intended to transform artworks from isolated visual objects into interpretable cultural artifacts with historical and educational value.

In addition to textual information, the system integrates multimedia interpretation features, including audio narration and high-resolution imagery. Audio guides allow visitors to experience guided storytelling and contextual explanations while physically viewing the artwork. To improve accessibility, transcripts are also provided alongside audio content, ensuring that users with hearing impairments or reading preferences can access the same information.

The application also provides high-quality digital images of artworks that support zooming and detailed inspection. This enables visitors to examine fine artistic details that may not be clearly visible in the physical exhibition environment.

To facilitate seamless interaction between the physical museum environment and the digital application, each artwork is associated with a unique identifier, such as a QR code or exhibit number. Users can scan QR codes placed near exhibits or manually enter exhibit identifiers to instantly retrieve the corresponding digital information. This minimizes interaction effort and enables rapid context-aware content access during museum exploration.

Furthermore, each artwork is linked to its physical location within the museum layout. This spatial association allows the system to display the artwork's position on an interactive digital map and supports navigation-related features throughout the application.

1.1.2 Search and Intelligent Exploration

The system supports multiple mechanisms for artwork discovery and exploration. Users may browse artworks manually through categorized interfaces or perform keyword-based searches using artwork titles, artist names, historical periods, artistic styles, or thematic concepts.

To further improve user interaction, the system incorporates intelligent retrieval capabilities through a conversational assistant component. Users may ask natural-language questions regarding artworks, artists, collections, or museum-related information. The assistant utilizes retrieval-based artificial intelligence techniques to generate contextually relevant responses based on the museum database and associated content resources.

This intelligent interaction model transforms the application from a static informational catalog into a dynamic and interactive learning companion. The assistant is designed to support educational exploration, contextual clarification, and personalized engagement throughout the museum visit.

1.1.3 Curated and Specialized Tours

The system supports curated tours that guide visitors through predefined sequences of artworks organized around specific themes, historical narratives, artistic movements, or cultural topics. These tours provide structured exploration experiences that encourage visitors to understand relationships between artworks rather than viewing exhibits in isolation.

Specialized tours may focus on themes such as colonial art, modernism, traditional artistic techniques, regional cultural heritage, or particular artistic schools. By following these curated experiences, users can gain a more coherent understanding of artistic development and conceptual relationships between exhibits.

Each tour includes an ordered list of artworks, estimated completion duration, and recommended navigation path throughout the museum space. The system may also provide personalized recommendations based on user preferences, previously viewed artworks, saved collections, or interaction history.

By integrating guided exploration with contextual interpretation, the tour system en-

hances both educational value and visitor engagement within the museum environment.

1.1.4 Collections and Personalized Content Organization

In addition to individual artworks and tours, the system organizes museum content into collections that group related artworks together according to thematic, historical, artistic, or cultural relationships.

Collections may be organized based on artist, historical period, artistic medium, cultural context, or conceptual similarity. This organizational structure provides visitors with alternative exploration pathways and supports deeper thematic understanding of museum content.

The system also supports personalized user interaction through bookmarking and saving functionalities. Users may create personal collections by saving artworks or tours that they find meaningful or interesting during their museum visit.

These personalized collections enable continued engagement beyond the physical museum experience by allowing users to revisit saved artworks, review previously explored tours, and maintain long-term interaction with cultural content.

1.1.5 Interactive Map and Location-Aware Navigation

A major component of the proposed system is the integration of an interactive digital map representing the physical layout of the museum. The map serves both as a navigation interface and as a contextual exploration tool.

Users can visualize exhibition rooms, artwork locations, tour routes, and nearby exhibits directly within the application. This reduces confusion within large or complex museum spaces and improves visitor orientation during exploration.

The system may utilize device positioning capabilities or indoor location approximation techniques to estimate the user's position within the museum. Based on spatial context, the application can highlight nearby artworks, recommend relevant exhibits, suggest navigation routes, or guide visitors through curated tours.

By combining spatial awareness with multimedia content delivery, the map transforms the application into both an informational resource and an intelligent navigation assistant.

1.1.6 User Accounts and Personalization

The system supports authenticated user accounts that enable personalized interaction and persistent user data management. Through secure authentication mechanisms, users may maintain saved artworks, bookmarked tours, interaction history, and personalized collections across multiple sessions.

Personalization features may include recommended tours, suggested artworks based on previous interactions, recently viewed exhibits, and user-specific exploration preferences. These features aim to provide more engaging and adaptive museum experiences tailored to individual visitor interests.

1.1.7 Summary

Overall, the proposed Museum Companion System functions as a comprehensive digital platform that enhances museum exploration through multimedia interpretation, intelligent assistance, personalized interaction, and spatial navigation.

By integrating artwork information, audio narration, intelligent retrieval, curated tours, thematic collections, interactive mapping, and user personalization into a unified application, the system transforms traditional museum visits into more immersive, accessible, and educational experiences. The application not only supports efficient navigation and information access but also encourages deeper engagement with cultural heritage through contextual understanding and interactive exploration.

Chapter 2

Requirement Analysis

2.1 Stakeholders

The proposed Museum Companion System involves several stakeholders who either directly interact with the application or are affected by its outcomes. These stakeholders are categorized into primary stakeholders and secondary stakeholders.

2.1.1 Primary Stakeholders

Visitors. Visitors are the main users of the system. They include tourists, students, researchers, art enthusiasts, and casual museum-goers. Visitors use the mobile application as a virtual companion during their museum visit.

Their main goals are to access artwork information, scan exhibit QR codes, listen to audio guides, search for artworks, follow curated tours, view museum maps, and save artworks or tours for later review. Visitors expect the system to be intuitive, responsive, visually clear, accessible, and helpful within the physical museum environment.

Museum Staff and Curators. Museum staff and curators are responsible for preparing and maintaining the cultural content presented in the system. This includes artwork descriptions, artist information, collection details, tour narratives, images, audio guides, transcripts, and exhibit location data.

They require a reliable content management process that allows museum information to remain accurate, organized, and up to date. Their main concern is that the application

represents artworks and museum knowledge correctly.

System Administrators. System administrators are responsible for managing the technical operation of the system. Their tasks include managing staff access, monitoring backend services, maintaining system security, and ensuring the availability of the application.

Their primary concerns are system stability, authentication, authorization, data protection, performance, and maintainability.

2.1.2 Secondary Stakeholders

Museum Management. Museum management is concerned with the strategic value of the system. Their interests include improving visitor engagement, increasing the museum's digital presence, supporting educational outreach, and understanding visitor behavior through system usage data.

Developers and Maintenance Team. Developers are responsible for implementing, extending, and maintaining the system. They require clear requirements, modular architecture, well-defined APIs, maintainable code, and extensibility for future features such as improved recommendation systems, multilingual support, or augmented reality.

External Service Providers. The system may rely on external services such as Firebase Authentication, cloud storage, location services, vector database services, and large language model APIs. These services indirectly affect the system's authentication, media delivery, location awareness, and intelligent assistant functionality.

2.2 Functional Requirements

The system shall provide functionalities that support museum exploration, artwork interpretation, personalization, navigation, content management, and intelligent assistance.

2.2.1 Artwork Information and Access

- **FR01.** The system shall allow visitors to view a list of artworks available in the museum.

- **FR02.** The system shall display detailed artwork information, including title, artist, year or period, medium, dimensions, description, image, location, and related metadata.
- **FR03.** The system shall provide interpretive descriptions that explain the historical, cultural, and artistic significance of artworks.
- **FR04.** The system shall display high-quality artwork images and support detailed visual inspection.
- **FR05.** The system shall provide audio narration for artworks when audio resources are available.
- **FR06.** The system shall provide transcripts for audio guides to support accessibility.
- **FR07.** The system shall allow visitors to access artwork information by browsing, searching, scanning QR codes, or entering exhibit identifiers.
- **FR08.** The system shall associate each artwork with a physical room or map location within the museum.

2.2.2 Search, QR Code, and Discovery

- **FR09.** The system shall allow visitors to search artworks by title, artist, keyword, period, medium, or theme.
- **FR10.** The system shall display search results with artwork images, titles, artists, and brief metadata.
- **FR11.** The system shall allow visitors to scan QR codes placed near physical exhibits.
- **FR12.** The system shall retrieve and display the matching artwork when a valid QR code or exhibit number is provided.
- **FR13.** The system shall display an appropriate message when no matching artwork is found.

2.2.3 Intelligent Assistant

- **FR14.** The system shall provide a conversational assistant that allows visitors to ask questions about artworks, artists, tours, collections, and museum content.
- **FR15.** The assistant shall retrieve relevant museum information before generating an answer.
- **FR16.** The assistant shall respond based on available museum data and indicate when requested information is unavailable.
- **FR17.** The system shall store assistant conversations when the user is authenticated, if conversation history is supported.

2.2.4 Tours and Collections

- **FR18.** The system shall provide curated tours based on themes, historical periods, artistic movements, techniques, or cultural narratives.
- **FR19.** The system shall display tour details, including title, description, estimated duration, and ordered artwork stops.
- **FR20.** The system shall allow visitors to follow a tour step by step.
- **FR21.** The system shall organize artworks into collections based on artist, period, medium, theme, or cultural relationship.
- **FR22.** The system shall allow visitors to browse collections and view artworks inside a selected collection.

2.2.5 Personalization and Saved Content

- **FR23.** The system shall allow visitors to sign in using Firebase Authentication.
- **FR24.** The system shall allow authenticated visitors to save artworks to a personal favorites list.
- **FR25.** The system shall allow authenticated visitors to save tours for later access.

- **FR26.** The system shall synchronize saved artworks and saved tours with the back-end service.
- **FR27.** The system shall provide personalized recommendations when sufficient user interaction data is available.

2.2.6 Interactive Map and Location Features

- **FR28.** The system shall provide a digital museum map showing rooms and artwork locations.
- **FR29.** The system shall allow visitors to view the location of a selected artwork on the map.
- **FR30.** The system shall highlight nearby artworks when location information is available.
- **FR31.** The system shall support navigation assistance between artworks, rooms, or tour stops.
- **FR32.** The system shall provide a fallback map browsing mode when precise location data is unavailable.

2.2.7 Content Management

- **FR33.** The system shall allow authorized staff to create, update, and delete artwork records.
- **FR34.** The system shall allow authorized staff to manage artist information.
- **FR35.** The system shall allow authorized staff to upload and manage artwork images and audio files.
- **FR36.** The system shall allow authorized staff to create and manage museum rooms, collections, and tours.
- **FR37.** The system shall validate required content fields before saving staff changes.

2.2.8 System and Data Synchronization

- **FR38.** The system shall retrieve museum data from backend APIs.
- **FR39.** The system shall cache previously loaded data locally using mobile persistence mechanisms.
- **FR40.** The system shall support offline access to previously loaded artwork, tour, and collection data.
- **FR41.** The system shall synchronize user-specific data when network connectivity becomes available.

2.3 Non-Functional Requirements

2.3.1 Performance

- **NFR01.** The system shall load common artwork detail pages within an acceptable response time under normal network conditions.
- **NFR02.** The system shall display cached content quickly when offline or under poor network conditions.
- **NFR03.** The backend shall support multiple concurrent visitors without significant degradation in response quality.

2.3.2 Usability

- **NFR04.** The mobile interface shall be intuitive and require minimal steps to access core features such as artwork details, QR scanning, search, tours, and map navigation.
- **NFR05.** The interface shall use a clear visual hierarchy suitable for museum exploration.
- **NFR06.** The system shall maintain a consistent visual identity aligned with the museum companion concept.

2.3.3 Reliability

- **NFR07.** The system shall handle network failures gracefully.
- **NFR08.** The system shall provide meaningful error messages when content, audio, images, or map data cannot be loaded.
- **NFR09.** The system shall avoid data loss when saving user favorites or tour progress during temporary connectivity issues.

2.3.4 Security

- **NFR10.** The system shall use Firebase Authentication for user identity management.
- **NFR11.** The backend shall verify authentication tokens before granting access to protected user or staff functions.
- **NFR12.** Administrative and staff functions shall require proper authorization.
- **NFR13.** Communication between the mobile application and backend services shall use secure network protocols.

2.3.5 Scalability

- **NFR14.** The system shall support an increasing number of artworks, artists, tours, collections, and users.
- **NFR15.** The architecture shall allow future integration of additional museums, recommendation methods, multilingual content, or augmented reality features.

2.3.6 Maintainability

- **NFR16.** The system shall be designed with modular frontend and backend components.
- **NFR17.** The backend shall separate concerns between artwork management, users, tours, collections, assistant functions, and recommendation logic.

- **NFR18.** The mobile application shall separate user interface, data models, persistence, and API communication logic.

2.3.7 Compatibility

- **NFR19.** The mobile application shall support modern iOS devices.
- **NFR20.** The backend APIs shall be designed to allow future support for other client platforms such as Android or web applications.

2.3.8 Accessibility

- **NFR21.** The system shall provide transcripts for audio content.
- **NFR22.** The system shall use readable typography and sufficient visual contrast.
- **NFR23.** The system should support multilingual content when museum data is available in multiple languages.

2.4 Use Cases

The Museum Companion System supports interactions between visitors, museum staff, system administrators, and external services. The primary actor is the *Visitor*, who uses the system during museum exploration. Supporting actors include *Museum Staff*, who manage museum content, and *System Administrator*, who manages access and technical configuration.

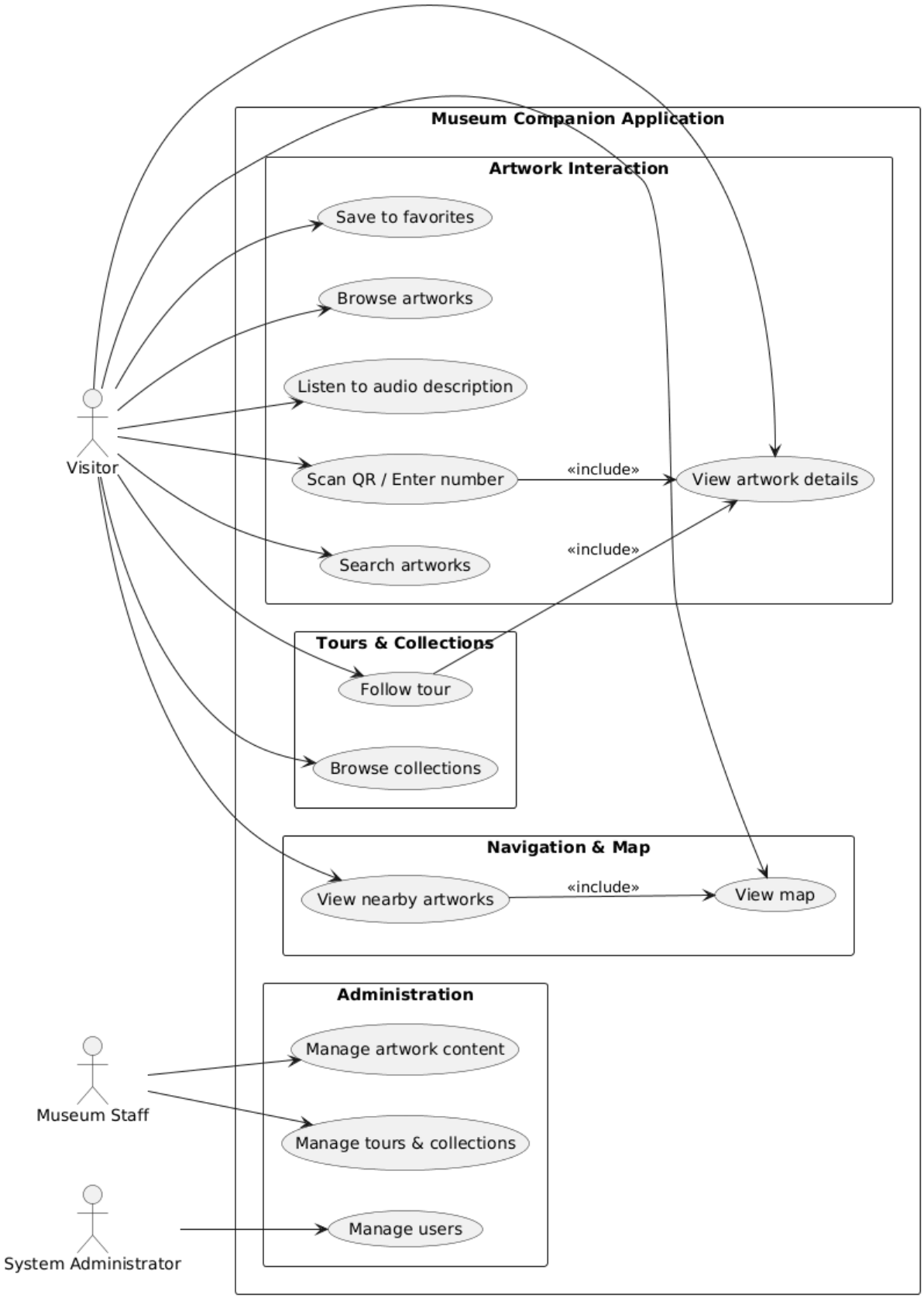


Figure 2.1: Use-case diagram of the Museum Companion System

2.4.1 Use Case List

- UC01 — Browse artworks
- UC02 — View artwork details
- UC03 — Listen to audio description
- UC04 — Scan QR code or enter artwork number
- UC05 — Search artworks
- UC06 — Ask the intelligent assistant
- UC07 — View artwork location on map
- UC08 — Follow a curated tour
- UC09 — Browse collections
- UC10 — Save artwork or tour
- UC11 — View nearby artworks
- UC12 — Manage artwork content
- UC13 — Manage tours and collections
- UC14 — Manage users and system access

2.4.2 Detailed Use Case Tables

UC01 — Browse artworks

Use Case ID	UC01
Name	Browse artworks
Primary Actor	Visitor
Description	The visitor browses the artworks available in the museum application.
Preconditions	Artwork data is available locally or through the backend service.
Postconditions	A list of artworks is displayed.
Main Flow	1. The visitor opens the application. 2. The system displays the home or browse screen. 3. The visitor selects the artwork browsing section. 4. The system retrieves artwork data. 5. The system displays the list of artworks.
Alternative Flows	1. If the network is unavailable, the system displays cached artwork data. 2. If no artwork data is available, the system displays an empty-state message.

UC02 — View artwork details

Use Case ID	UC02
Name	View artwork details
Primary Actor	Visitor
Description	The visitor opens a specific artwork page to view detailed information.
Preconditions	The visitor has selected an artwork from browsing, searching, scanning, a tour, or a collection.
Postconditions	The artwork detail page is displayed.
Main Flow	1. The visitor selects an artwork. 2. The system retrieves the artwork record. 3. The system displays the artwork title, artist, image, year, medium, description, location, and available actions.
Alternative Flows	1. If the artwork cannot be retrieved, the system displays an error message. 2. If only cached data is available, the system displays the available offline content.

UC03 — Listen to audio description

Use Case ID	UC03
Name	Listen to audio description
Primary Actor	Visitor
Description	The visitor listens to audio narration for a selected artwork.
Preconditions	The selected artwork contains an audio guide.
Postconditions	The audio guide is played, paused, or stopped according to visitor interaction.
Main Flow	1. The visitor opens an artwork detail page. 2. The visitor selects the audio guide option. 3. The system loads the audio file. 4. The system begins playback. 5. The system displays transcript content when available.
Alternative Flows	1. If the audio file is unavailable, the system notifies the visitor. 2. If the visitor pauses playback, the system preserves the current playback position.

UC04 — Scan QR code or enter artwork number

Use Case ID	UC04
Name	Scan QR code or enter artwork number
Primary Actor	Visitor
Description	The visitor directly accesses an artwork by scanning a QR code or entering an exhibit number.
Preconditions	The artwork has a valid QR code or exhibit identifier stored in the system.
Postconditions	The matching artwork detail page is opened.
Main Flow	1. The visitor opens the scan or manual entry feature. 2. The visitor scans a QR code or enters an artwork number. 3. The system validates the input. 4. The system retrieves the matching artwork. 5. The system displays the artwork detail page.
Alternative Flows	1. If the QR code or number is invalid, the system displays a not-found message. 2. If camera permission is denied, the system offers manual entry.

UC05 — Search artworks

Use Case ID	UC05
Name	Search artworks
Primary Actor	Visitor
Description	The visitor searches for artworks using keywords or categories.
Preconditions	The visitor is on the search interface.
Postconditions	Matching artworks are displayed.
Main Flow	1. The visitor opens the search feature. 2. The visitor enters a keyword. 3. The system searches artwork records. 4. The system displays matching results. 5. The visitor selects a result to view details.
Alternative Flows	1. If no results are found, the system displays a no-results message. 2. If the query is empty, the system may display suggestions or popular artworks.

UC06 — Ask the intelligent assistant

Use Case ID	UC06
Name	Ask the intelligent assistant
Primary Actor	Visitor
Description	The visitor asks a natural-language question about artworks, artists, collections, tours, or museum content.
Preconditions	The assistant service is available.
Postconditions	The visitor receives a contextual response based on available museum data.
Main Flow	1. The visitor opens the assistant interface. 2. The visitor enters a question. 3. The system retrieves relevant museum information. 4. The assistant generates a response. 5. The system displays the response to the visitor.
Alternative Flows	1. If relevant museum information is unavailable, the assistant informs the visitor that the requested information is not available. 2. If the assistant service fails, the system displays an error message.

UC07 — View artwork location on map

Use Case ID	UC07
Name	View artwork location on map
Primary Actor	Visitor
Description	The visitor views the physical location of an artwork on the museum map.
Preconditions	The selected artwork has location data.
Postconditions	The artwork location is highlighted on the map.
Main Flow	1. The visitor opens an artwork detail page. 2. The visitor selects the map option. 3. The system opens the museum map. 4. The system highlights the artwork location.
Alternative Flows	1. If location data is unavailable, the system notifies the visitor. 2. If the map cannot be loaded, the system displays a fallback message.

UC08 — Follow a curated tour

Use Case ID	UC08
Name	Follow a curated tour
Primary Actor	Visitor
Description	The visitor follows a curated tour based on a theme, period, style, or historical narrative.
Preconditions	At least one tour is available.
Postconditions	The visitor is guided through the ordered artworks in the tour.
Main Flow	1. The visitor opens the tours section. 2. The system displays available tours. 3. The visitor selects a tour. 4. The system displays tour details, duration, and stops. 5. The visitor starts the tour. 6. The system guides the visitor through the ordered stops.
Alternative Flows	1. If no tour is available, the system displays an empty-state message. 2. If the visitor exits early, the system may save tour progress.

UC09 — Browse collections

Use Case ID	UC09
Name	Browse collections
Primary Actor	Visitor
Description	The visitor explores collections of related artworks.
Preconditions	Collection data exists in the system.
Postconditions	The visitor views artworks inside a selected collection.
Main Flow	1. The visitor opens the collections section. 2. The system displays available collections. 3. The visitor selects a collection. 4. The system displays collection details and related artworks.
Alternative Flows	1. If the collection is empty, the system displays an empty-state message.

UC10 — Save artwork or tour

Use Case ID	UC10
Name	Save artwork or tour
Primary Actor	Visitor
Description	The visitor saves an artwork or tour for later access.
Preconditions	The visitor is viewing an artwork or tour.
Postconditions	The selected item is saved to the visitor's account or local storage.
Main Flow	1. The visitor opens an artwork or tour page. 2. The visitor selects the save option. 3. The system stores the selected item. 4. The system confirms the save action.
Alternative Flows	1. If the visitor is not signed in, the system may request sign-in or save locally. 2. If synchronization fails, the system stores the action locally for later synchronization.

UC11 — View nearby artworks

Use Case ID	UC11
Name	View nearby artworks
Primary Actor	Visitor
Description	The visitor uses the map or location feature to discover nearby artworks.
Preconditions	Map data is available. Location permission may be required for automatic positioning.
Postconditions	Nearby artworks or the default museum map are displayed.
Main Flow	1. The visitor opens the map or nearby feature. 2. The system requests location permission if needed. 3. The visitor grants permission. 4. The system determines the approximate location. 5. The system displays nearby artworks.
Alternative Flows	1. If permission is denied, the system allows manual map browsing. 2. If location cannot be determined, the system displays the default museum map.

UC12 — Manage artwork content

Use Case ID	UC12
Name	Manage artwork content
Primary Actor	Museum Staff
Description	Authorized museum staff create, edit, or delete artwork information and media.
Preconditions	Staff is authenticated and authorized.
Postconditions	Artwork content is updated in the system.
Main Flow	1. Staff logs into the management system. 2. Staff opens the artwork management section. 3. Staff creates, edits, or deletes an artwork entry. 4. Staff updates text, images, audio, QR mapping, and location data. 5. The system validates and saves the changes.
Alternative Flows	1. If validation fails, the system displays input errors. 2. If media upload fails, the system preserves existing content and requests retry.

UC13 — Manage tours and collections

Use Case ID	UC13
Name	Manage tours and collections
Primary Actor	Museum Staff
Description	Authorized staff create, update, and organize tours and collections.
Preconditions	Staff is authenticated and authorized.
Postconditions	Tours and collections are stored or updated.
Main Flow	1. Staff opens the tours and collections management feature. 2. Staff creates or edits a tour or collection. 3. Staff selects artworks to include. 4. Staff defines title, description, ordering, duration, and metadata. 5. The system validates and saves the configuration.
Alternative Flows	1. If an artwork reference is invalid, the system rejects the change and reports the issue.

UC14 — Manage users and system access

Use Case ID	UC14
Name	Manage users and system access
Primary Actor	System Administrator
Description	The administrator manages staff accounts, permissions, and system access policies.
Preconditions	The administrator is authenticated and authorized.
Postconditions	User permissions and access settings are updated.
Main Flow	1. The administrator logs into the administration interface. 2. The administrator opens the access management section. 3. The administrator creates, edits, disables, or removes accounts. 4. The administrator assigns or updates permission roles. 5. The system saves the changes.
Alternative Flows	1. If an invalid role assignment is attempted, the system rejects the change. 2. If an account cannot be removed due to dependency constraints, the system notifies the administrator.

2.5 Context Diagram

A context diagram presents the system at the highest level of abstraction. It treats the Museum Companion System as a single process and focuses on the interactions between

the system and external entities.

The purpose of the context diagram is to clarify the system boundary and identify the main sources of input and output. It does not describe internal processes in detail. Instead, it emphasizes what data enters the system, what data leaves the system, and which external entities are involved.

For the proposed Museum Companion System, the following external entities are identified:

- **Visitor**: the main end user who browses artworks, scans QR codes, searches content, listens to audio descriptions, follows tours, saves artworks, interacts with the assistant, and uses the map.
- **Museum Staff**: authorized personnel who manage artwork information, artist records, media resources, rooms, tours, and collections.
- **System Administrator**: personnel responsible for managing accounts, permissions, system configuration, and access control.
- **Authentication Service**: an external service, such as Firebase Authentication, used to verify user identity.
- **Location Service**: an external service or device capability that provides approximate location information for map and nearby-artwork features.
- **AI and Retrieval Services**: external or supporting services used by the intelligent assistant to retrieve relevant information and generate contextual responses.

The *Visitor* sends requests such as search queries, QR scan inputs, artwork number entries, tour selections, saved-content actions, assistant questions, and map interactions. In return, the system provides artwork details, images, audio guides, transcripts, tour information, collection content, map positions, recommendations, and assistant responses.

The *Museum Staff* provides content management input, including artwork records, edited descriptions, uploaded images, audio files, transcripts, room data, tours, and collections. The system returns validation messages, confirmation responses, and updated content states.

The *System Administrator* provides account management and access-control actions. The system responds with permission updates, administrative status information, and system configuration results.

The *Authentication Service* verifies login credentials and returns identity tokens. The system uses these tokens to identify users and protect restricted features.

The *Location Service* supplies approximate position data used for map-based interaction, nearby artwork discovery, and tour guidance.

The *AI and Retrieval Services* support the assistant by retrieving relevant museum knowledge and generating responses based on available system data.

Figure 2.2 presents the context diagram of the Museum Companion System.

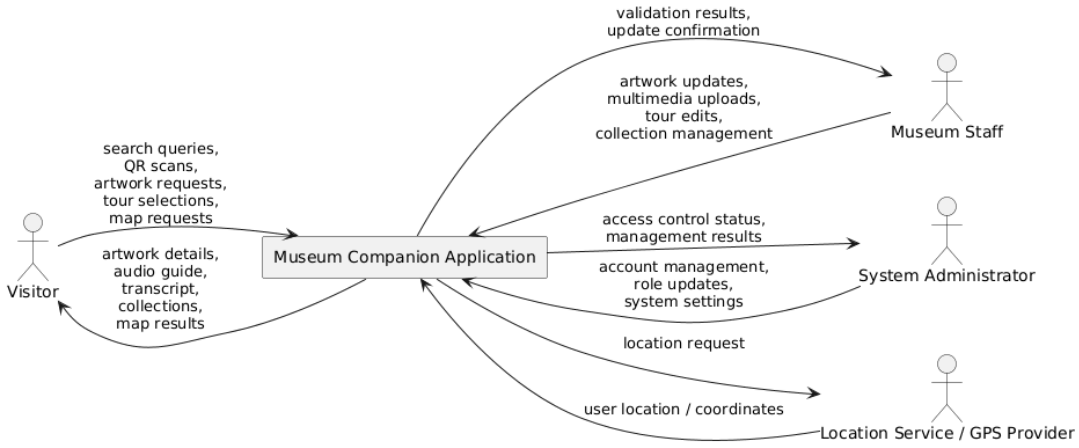


Figure 2.2: Context diagram of the Museum Companion System

The context diagram shows that the Museum Companion System acts as a central mediator between visitors, museum personnel, system administrators, authentication services, location services, and AI-supported retrieval services. This highlights that the application is not merely a static information viewer, but a digital platform supporting exploration, personalization, administration, and intelligent interaction.

2.6 Data Flow Diagram

While the context diagram provides a high-level external view of the system, the data flow diagram provides a more detailed representation of how data moves between processes, external entities, and data stores inside the system.

In this project, a Level 0 Data Flow Diagram is used to decompose the Museum Companion System into its principal processes. These processes capture the major functional responsibilities of the system while remaining abstract enough for requirements analysis.

The main external entities are:

- Visitor
- Museum Staff
- System Administrator
- Authentication Service
- Location Service
- AI and Retrieval Services

The system is divided into the following major processes:

1. **Artwork Information and Media Access**: retrieves and presents artwork details, metadata, images, audio guides, transcripts, artist information, and room information.
2. **Search, Scan, and Discovery**: processes keyword searches, QR code scans, and exhibit number input to locate relevant artworks.
3. **Assistant and Recommendation**: handles visitor questions, retrieves relevant museum knowledge, generates assistant responses, and provides recommendations when applicable.
4. **Tours and Collections**: manages the retrieval and presentation of curated tours, thematic collections, and artwork groupings.
5. **Map and Location Services**: manages museum map display, artwork positioning, nearby artwork suggestions, and location-based navigation support.
6. **User Profile and Saved Content**: manages authenticated user profiles, saved artworks, saved tours, and personalization data.

7. **Administrative and Content Management**: supports staff and administrators in creating, updating, validating, and organizing content and access permissions.

The DFD includes the following internal data stores:

- **Artwork Data Store**: stores artworks, artists, descriptions, metadata, QR identifiers, images, audio references, transcripts, and room associations.
- **Tours and Collections Data Store**: stores curated tours, tour-artwork relationships, collections, and collection-artwork relationships.
- **User Data Store**: stores user profiles, saved artworks, saved tours, and personalization-related data.
- **Conversation Data Store**: stores assistant conversations and messages when conversation history is supported.
- **Administration Data Store**: stores staff accounts, roles, permissions, and administrative configuration.
- **Vector Knowledge Store**: stores embedded museum knowledge used by the retrieval component of the intelligent assistant.

In the Level 0 DFD, the *Visitor* interacts with artwork access, search and scan, assistant, tours and collections, map services, and saved-content processes. For example, when a visitor scans a QR code, the input is processed by the *Search, Scan, and Discovery* process, which queries the artwork data store and returns the matching artwork page.

When the visitor asks a question, the *Assistant and Recommendation* process retrieves relevant information from the vector knowledge store and structured museum data, then returns a contextual response. When the visitor saves an artwork or tour, the *User Profile and Saved Content* process stores the action in the user data store.

The *Museum Staff* interacts with the *Administrative and Content Management* process to update artwork records, media files, tours, collections, and room information. The *System Administrator* uses the same administrative process to manage roles, permissions, and access policies.

The *Authentication Service* supports identity verification for visitors, staff, and administrators. The *Location Service* provides approximate position data for map-based functions. The *AI and Retrieval Services* support intelligent responses and contextual recommendations.

Figure 2.3 presents the Level 0 Data Flow Diagram of the Museum Companion System.

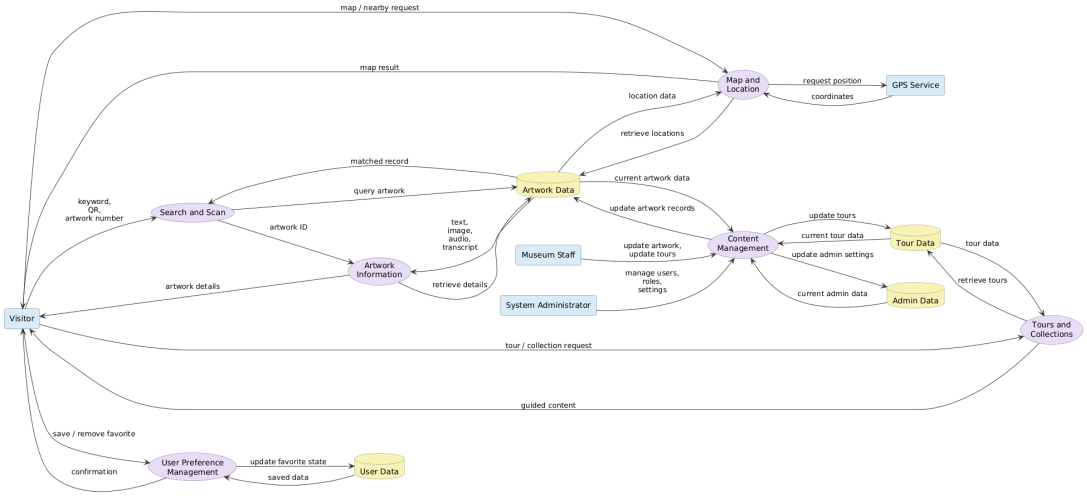


Figure 2.3: Level 0 Data Flow Diagram of the Museum Companion System

2.6.1 Interpretation of the Data Flow Diagram

The DFD reveals several important characteristics of the proposed system.

First, the system is highly *content-driven*. Most visitor-facing functions depend on structured museum data, including artwork descriptions, artist information, multimedia resources, location data, tours, and collections. Therefore, data quality and consistency are essential to the usefulness of the application.

Second, the system is *interaction-oriented*. Visitors do not only read static content; they search, scan, ask questions, navigate, save items, and follow guided experiences. The system must therefore provide smooth transitions between physical exhibits and digital interactions.

Third, the system is *personalization-aware*. Saved artworks, saved tours, user profiles, and assistant history allow the application to support continued engagement beyond a single museum visit.

Fourth, the system is *administratively supported*. Museum staff and administrators

are required to maintain accurate content, update tours and collections, manage user access, and ensure that the system remains reliable.

Finally, the system is partially *location-aware*. By incorporating map and location-related processes, the application can provide contextual assistance based on the visitor's position inside or near the museum. This improves usability, especially in complex museum layouts.

2.6.2 Summary

In summary, the context diagram and Level 0 Data Flow Diagram complement each other. The context diagram defines the system boundary and external interactions, while the Data Flow decomposes the system into major internal processes and data stores. Together, they provide a clear analytical foundation for later design activities, including architecture modeling, database design, API planning, user interface design, and implementation.

2.7 Market Research

The museum and cultural heritage sector has increasingly adopted digital technologies to improve visitor engagement, accessibility, and educational value. Modern visitors, especially younger generations, often expect museums to provide interactive digital experiences rather than relying solely on static exhibition labels or traditional guided tours. This trend has led to the development of mobile museum companion applications that integrate multimedia content, QR-code interaction, digital navigation, and personalized exploration.

The market research for this project focuses on analyzing existing museum companion applications and identifying their strengths, limitations, and opportunities for improvement. Several case studies were examined, including *iMuseum VFA*, *Bảo Tàng Lịch Sử TP.HCM*, and *Smartify*. These applications provide useful insights into current design approaches and common challenges in digital museum experiences.

2.7.1 Target Audience Analysis

The proposed system targets a broad range of museum visitors, including students, young adults, adults, seniors, and disadvantaged users. Different user groups have different motivations and usability expectations. Students often prioritize educational value and practicality, while young adults tend to value aesthetics, social interaction, and ease of use. Older visitors may prefer simplicity and readability, whereas disadvantaged users require stronger accessibility support and inclusive design considerations.

This diversity indicates that the application must balance usability, accessibility, educational value, and visual appeal. As a result, the proposed system emphasizes inclusive design principles, readable interfaces, multimedia accessibility, and intuitive navigation.

2.7.2 Identified Problems in Current Museum Experiences

The research identified several recurring problems in existing museum experiences and museum applications.

First, visitors often have limited access to contextual information about artworks. Traditional museum labels usually provide only short descriptions, making it difficult for visitors to fully understand the historical, artistic, or cultural significance of exhibits.

Second, many museum visits remain passive experiences. Visitors typically walk through exhibitions without personalized guidance, interactive interpretation, or adaptive exploration support. Existing systems frequently provide static content rather than interactive engagement.

Third, language barriers and accessibility limitations reduce inclusiveness for many visitors. Some museum systems lack multilingual support, transcripts, accessibility-focused layouts, or adaptive content presentation.

Finally, navigation inside museums can be difficult, especially in large or architecturally complex spaces such as the Ho Chi Minh City Museum of Fine Arts. Visitors may struggle to locate exhibits, understand exhibition layouts, or follow coherent viewing paths.

These issues collectively suggest the need for a more integrated and intelligent museum companion system.

2.7.3 Case Study: iMuseum VFA

The *iMuseum VFA* application serves as the official digital companion for the Vietnam National Fine Arts Museum. The system supports QR-code-based artwork interaction and structured browsing through collections.

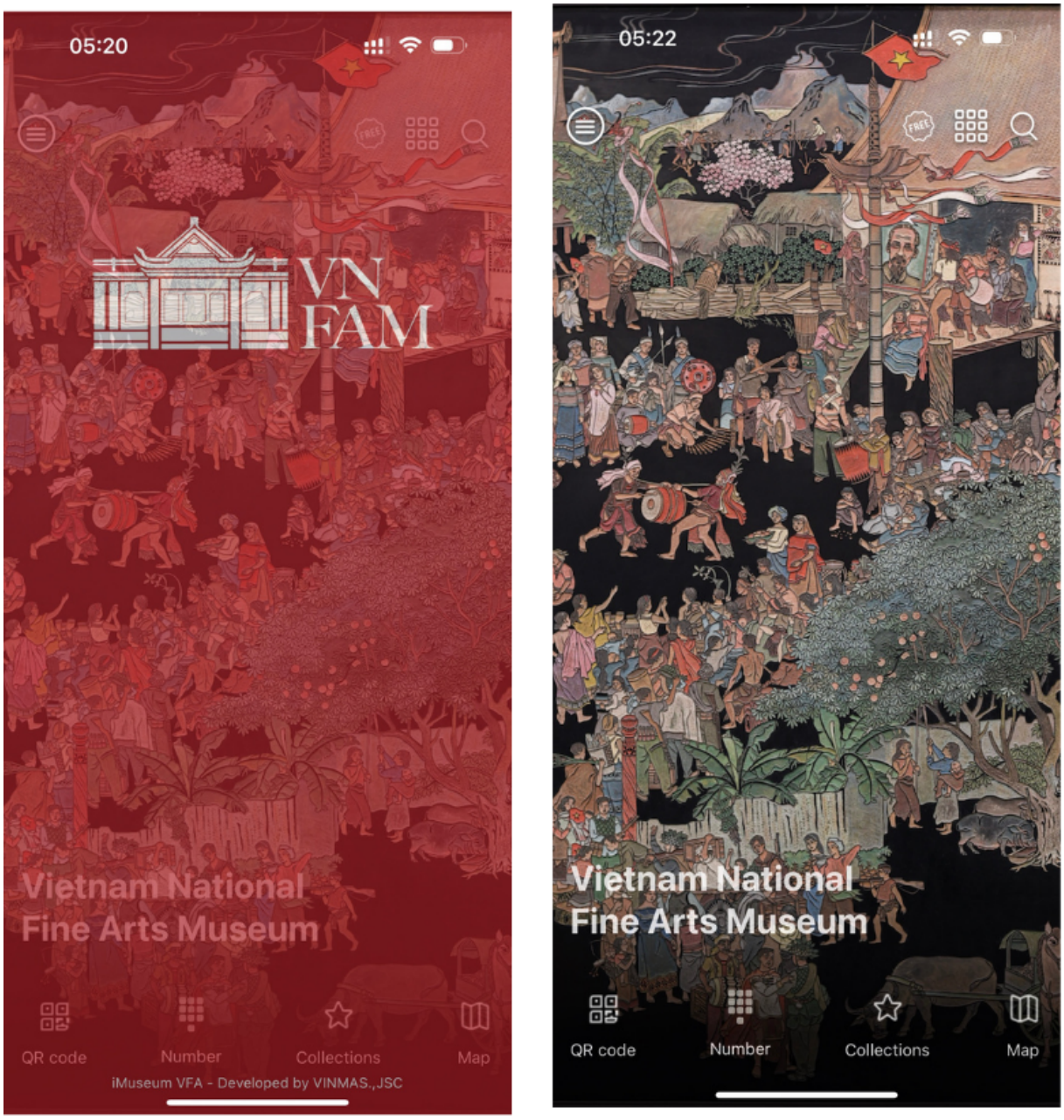


Figure 2.4: iMuseum VFA's basic UI

The application demonstrates several strengths. Its QR-code mechanism enables quick access to artwork information, while multimedia content such as text descriptions and audio guides supports self-guided museum exploration.

However, several limitations were identified during analysis. The application offers limited personalization features, uses a relatively primitive user interface design, and provides unintuitive navigation flows. Accessibility support is also minimal.

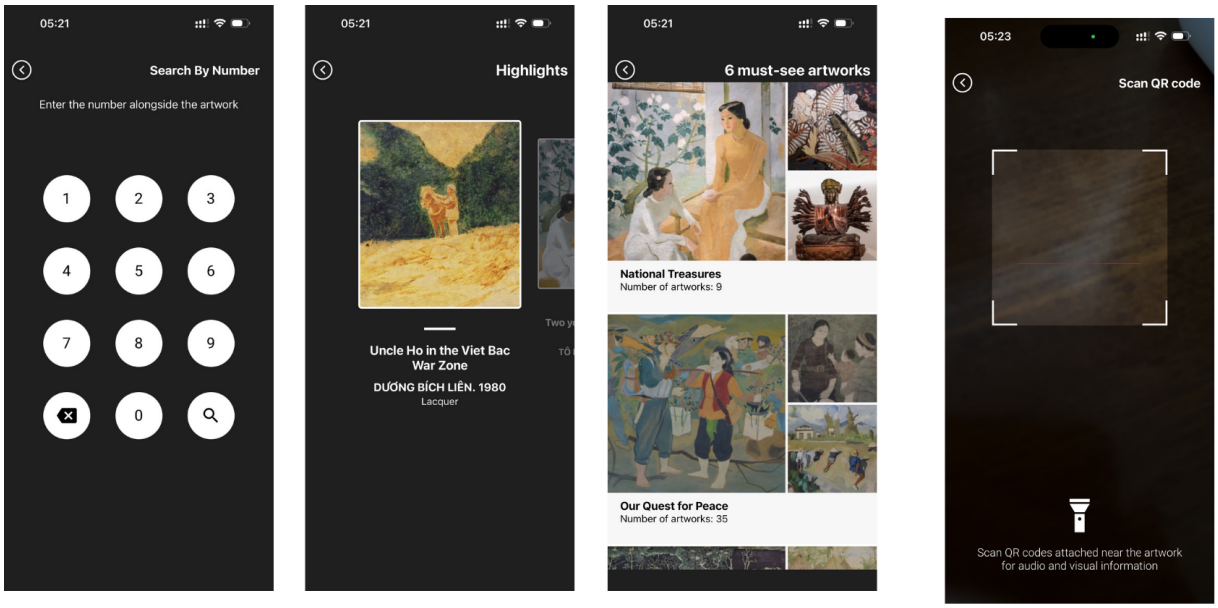


Figure 2.5: iMuseum VFA's basic operations

These limitations indicate opportunities for improvement in usability, personalization, accessibility, and interface design.

2.7.4 Case Study: Bảo Tàng Lịch Sử TP.HCM

The *Bảo Tàng Lịch Sử TP.HCM* application functions as a digital companion for the Ho Chi Minh City History Museum. Similar to iMuseum VFA, it supports QR-code interaction and museum storytelling experiences.



Figure 2.6: Basic operations of Bảo Tàng Lịch Sử TPHCM

The application demonstrates strengths in its clean and minimalist interface, integration of stories and collections, and relatively simple interaction workflow.

Nevertheless, the analysis also identified several shortcomings. The application lacks personalization, provides limited navigation support for stories and tours, and offers insufficient accessibility features such as responsive design and adaptive interface modes.

These observations reinforce the importance of balancing simplicity with navigation support, accessibility, and adaptive user interaction.

2.7.5 Case Study: Smartify

Smartify is a global museum companion platform that supports museums and galleries worldwide. The application includes artwork recognition, QR-code interaction, rich multimedia content, curated tours, and recommendation systems.

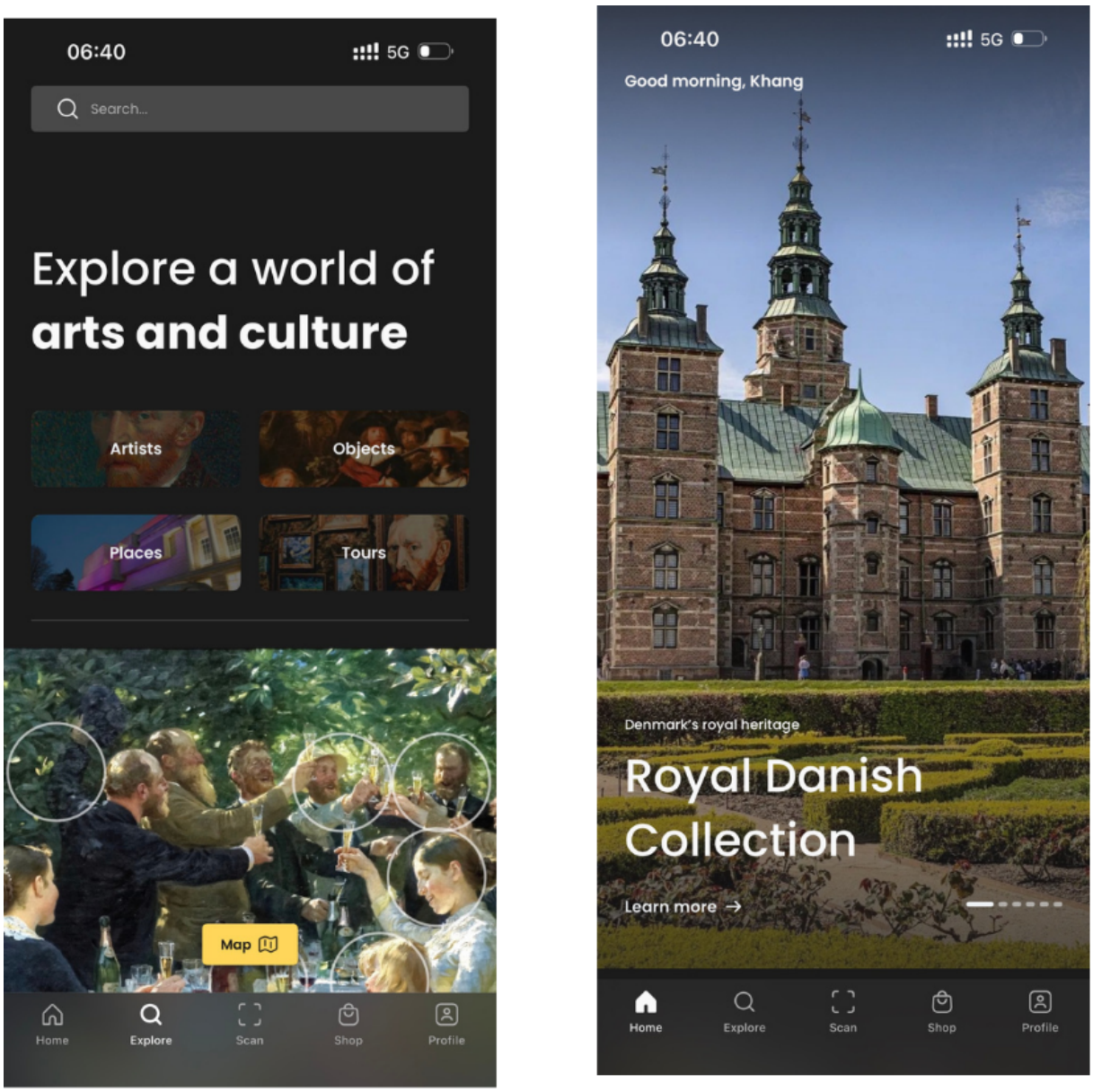


Figure 2.7: Smartify UI

Compared to the previous case studies, Smartify provides a more advanced multimedia and personalized experience. Its strengths include a clean interface, image-based artwork recognition, and recommendation-supported exploration.

However, Smartify also presents limitations. Many features require internet connectivity, some content is restricted behind premium access, and the system is not strongly localized for specific museum environments. In addition, because the platform is generalized across many institutions, it may be less tailored to the context, layout, and curatorial structure of a specific museum.

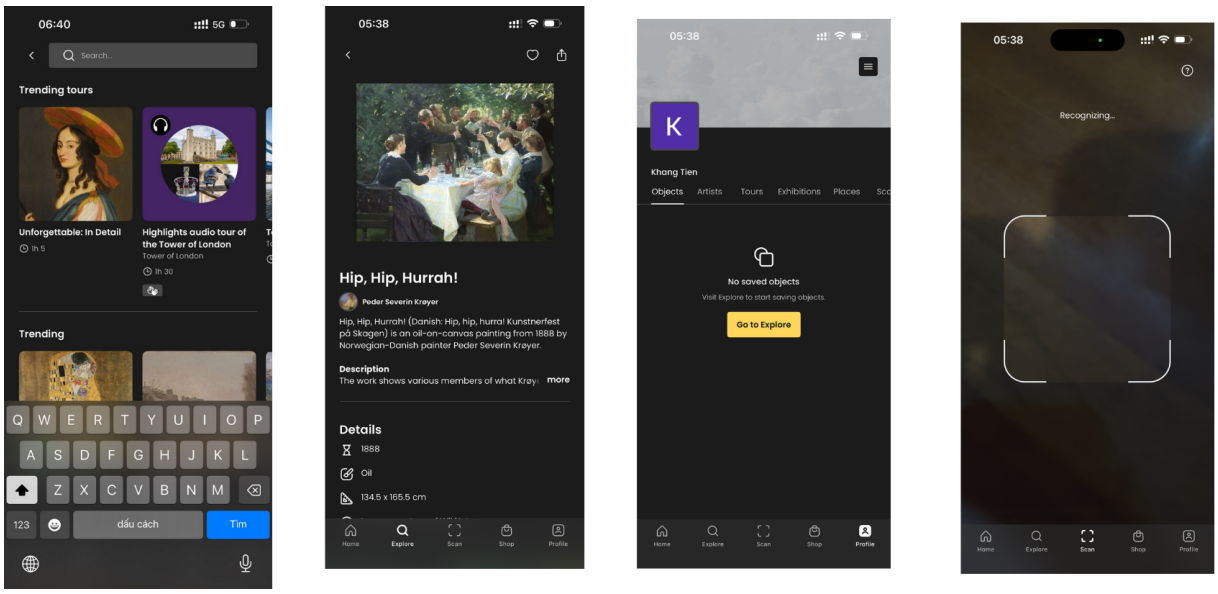


Figure 2.8: Smartify UI is considered more minimalistic but feature-rich

This suggests that localized museum-specific systems may provide more coherent navigation and contextual relevance for visitors.

2.7.6 Design Implications for the Proposed System

The proposed application aims to combine the strengths of existing systems while addressing their major limitations. Similar to existing museum applications, the system supports QR-code interaction, multimedia artwork interpretation, curated tours, and collections. However, the proposed system additionally emphasizes:

- Inclusive and accessibility-oriented design.
- Improved navigation and location-aware interaction.
- Personalized exploration and saved content.
- Intelligent assistant integration using retrieval-based AI.
- A balance between aesthetics and usability.
- Museum-specific contextual adaptation.

The design philosophy of the application is strongly influenced by Nielsen's usability heuristics and inclusive design principles. The system prioritizes readability, recognition-based navigation, clear visual hierarchy, and reduced cognitive load.

The interface design also incorporates museum-inspired aesthetics using colors such as ivory white, maroon, soft black, and Indochine yellow to create a visual identity aligned with the architecture and atmosphere of the Ho Chi Minh City Museum of Fine Arts.

2.7.7 Competitive Positioning

Compared to the analyzed applications, the proposed Museum Companion System positions itself as a more integrated and intelligent museum-specific platform.

Unlike simpler QR-based guide applications, the proposed system combines multimedia interpretation, interactive maps, curated tours, personalization, and conversational AI assistance within a unified experience.

Compared to broader global platforms such as Smartify, the proposed system focuses more strongly on localized museum adaptation, navigation support, inclusive design, and contextual interaction tailored to the physical museum environment.

Additionally, the integration of retrieval-augmented AI and recommendation mechanisms provides opportunities for more dynamic and personalized visitor experiences.

2.7.8 Summary

In summary, the market research demonstrates that digital museum companion systems are increasingly important in modern cultural experiences. Existing systems successfully provide QR-code interaction, multimedia guides, and digital content access, but many still suffer from limitations related to personalization, navigation, accessibility, and contextual interaction.

The proposed Museum Companion System addresses these limitations by integrating intelligent assistance, location-aware navigation, curated exploration, inclusive design, and multimedia interpretation into a unified application. As a result, the system aims to provide a more engaging, accessible, and educational museum experience compared to many existing solutions.

Chapter 3

UI-UX Design

3.1 Introduction

The user interface and user experience design of the proposed Museum Companion System play a critical role in determining the effectiveness, accessibility, and overall quality of the visitor experience. Unlike conventional mobile applications designed for productivity or entertainment alone, museum companion systems operate within a unique context where users simultaneously interact with both a digital interface and a physical cultural environment.

The application is therefore designed not merely as a standalone software product, but as an extension of the museum experience itself. Its role is to support visitors in understanding artworks, navigating exhibition spaces, discovering thematic relationships between exhibits, and interacting with cultural content in a more immersive and meaningful manner.

The design process focuses on reducing friction between physical exploration and digital interaction. Visitors should be able to access information quickly without interrupting their engagement with the museum environment. As a result, the application emphasizes intuitive navigation, recognition-based interaction, accessibility, readability, and aesthetically consistent presentation.

In addition to usability concerns, the application also aims to preserve the atmosphere and identity of the museum. The visual language of the interface is inspired by museum architecture, gallery environments, and cultural aesthetics associated with the Ho Chi

Minh City Museum of Fine Arts. Rather than adopting overly modern or visually aggressive design styles, the application uses calm colors, elegant typography, and balanced layouts to create a refined digital experience aligned with the museum context.

3.2 Design Objectives

The primary objective of the UI-UX design is to create an accessible and engaging digital companion that enhances museum exploration while remaining simple and intuitive for visitors of different backgrounds and age groups.

The interface is designed according to the following objectives:

- Provide fast and intuitive access to artwork information.
- Reduce cognitive load during museum exploration.
- Support accessibility and inclusive interaction.
- Balance aesthetics with usability.
- Create a visual identity aligned with museum culture and architecture.
- Support multimedia interaction without overwhelming users.
- Maintain consistency across all screens and interaction flows.
- Enable efficient navigation in large museum environments.
- Encourage exploration and long-term engagement with cultural content.

The system is intended to support both casual visitors and users seeking deeper educational or artistic understanding. Therefore, the design must accommodate different levels of familiarity with technology, different exploration goals, and different accessibility needs.

3.3 Target User Considerations

The application targets a broad and diverse audience, including students, young adults, tourists, adults, seniors, and disadvantaged users. Each user group interacts with museum environments differently and therefore requires different interface considerations.

3.3.1 Students

Students often visit museums for educational purposes and usually prioritize practicality and ease of use. They may interact with the application during guided educational activities or self-directed learning sessions. As a result, the interface must provide quick information retrieval, clear navigation, and easily understandable interaction flows.

Educational users may also require contextual explanations, structured tours, and guided interpretation to better understand artistic and historical concepts.

3.3.2 Young Adults

Young adults typically combine educational exploration with social and entertainment-oriented interaction. This demographic often values visual aesthetics, responsive interaction, smooth animations, and modern interface presentation.

The application therefore incorporates visually refined layouts, clean navigation patterns, multimedia support, and personalized interaction features such as saved artworks and recommendations.

3.3.3 Adults and Senior Visitors

Adult and senior visitors often prioritize readability, simplicity, and low interaction complexity. For these users, cluttered interfaces and overly complex navigation structures can significantly reduce usability.

The system addresses these concerns by emphasizing:

- Clear typography.
- Large touch targets.

- Predictable navigation.
- Strong visual hierarchy.
- Reduced interaction steps.

The interface also avoids excessive decorative elements that could distract users from primary functionality.

3.3.4 Disadvantaged Users and Accessibility Needs

Inclusive design is an important consideration because museums serve visitors with varying physical, sensory, and cognitive abilities.

The application therefore supports accessibility-oriented features such as:

- Audio narration.
- Text transcripts.
- High contrast presentation.
- Dark mode support.
- Readable typography.
- Responsive layouts.
- Multilingual support.

The interface is also designed to minimize cognitive overload by presenting information progressively rather than overwhelming users with excessive detail on a single screen.

3.4 Design Philosophy

The overall design philosophy of the system is strongly influenced by usability theory, inclusive design practices, and museum-centered interaction principles.

Two major conceptual foundations guide the design:

- Nielsen's usability heuristics.
- Inclusive design principles.

3.4.1 Usability-Oriented Design

The interface prioritizes usability over visual complexity. Visitors should be able to understand the system quickly without requiring tutorials or extensive learning.

Recognition-based interaction is emphasized throughout the application. Users should visually identify actions, navigation paths, and content structures without needing to memorize workflows or hidden functionality.

The system also emphasizes consistency. Similar interaction patterns, layouts, typography scales, and navigation structures are maintained across screens to reduce cognitive effort and improve learnability.



Figure 3.1: Colors for aesthestics and usability

3.4.2 Inclusive Design Principles

The application aims to support the widest possible audience rather than optimizing only for technologically experienced users.

Inclusive design considerations influence many aspects of the interface, including:

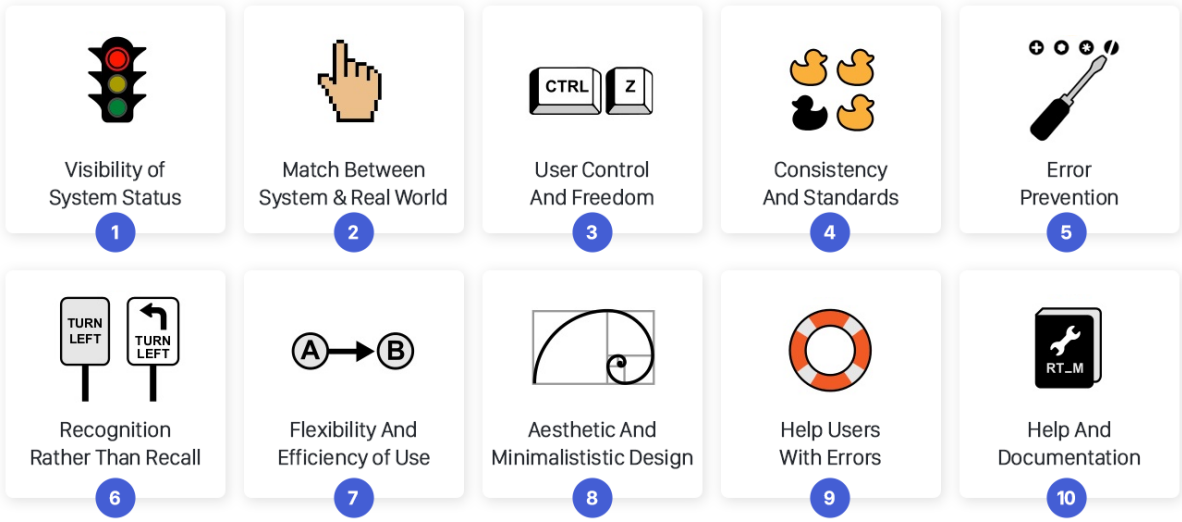


Figure 3.2: Nielsen's Heuristics

- Layout simplicity.
- Contrast and readability.
- Accessibility support.
- Responsive scaling.
- Multimedia alternatives.
- Navigation clarity.

These principles are particularly important in museum environments because visitors may vary significantly in age, education level, language familiarity, and physical accessibility needs.

3.4.3 Museum-Centered Visual Design

Unlike generic mobile applications, the Museum Companion System attempts to preserve the atmosphere and cultural identity of the museum itself.

The visual design intentionally avoids highly saturated colors, excessive visual noise, or overly playful interaction patterns. Instead, the application adopts calm and elegant visual presentation inspired by museum architecture and exhibition spaces.

The goal is for the application to feel like a natural extension of the museum environment rather than a separate digital product competing for user attention.

3.5 Color System

The color palette of the application is heavily inspired by the architecture and atmosphere of the Ho Chi Minh City Museum of Fine Arts.

The system primarily uses the following colors:

- Ivory White (#F8F4EC)
- Off-White (#F7F3EE)
- Soft Black (#171312)
- Maroon (#8B1D2F)
- Indochine Yellow (#C6A458)

3.5.1 Background and Neutral Colors

Ivory White and Off-White are used as primary background colors because they create a gallery-like visual atmosphere while maintaining readability and visual comfort.

These warm neutral tones also reduce the harshness often associated with pure white interfaces, creating a softer and more elegant presentation.

3.5.2 Primary Text and Contrast

Soft Black is used for typography and major interface elements. The contrast between Soft Black and the light background colors improves readability while maintaining a refined visual appearance.

The use of softer dark tones instead of pure black contributes to visual comfort during extended usage sessions.

3.5.3 Accent Colors

Maroon functions as the primary branding and accent color. It reflects classical museum aesthetics and architectural elements found in historical buildings.

Indochine Yellow is used selectively for highlighted actions, interactive emphasis, and dark mode contrast. Warm yellow tones create visual distinction without introducing aggressive color contrast.

3.5.4 Dark Mode

The system also supports dark mode to improve usability in low-light environments and accommodate different user preferences.

In dark mode, Indochine Yellow becomes more visually prominent because warm tones separate effectively from dark backgrounds while maintaining readability.

3.6 Typography

Typography is one of the most important visual components of the application because museum systems rely heavily on reading and interpretation.

The system uses two primary typefaces:

- Playfair Display.
- Inter.

3.6.1 Playfair Display

Playfair Display is primarily used for:

- Artwork titles.
- Major headings.
- Collection names.
- Hero sections.

The serif structure of Playfair Display conveys elegance, classical identity, and artistic atmosphere suitable for museum environments.

3.6.2 Inter

Inter is used for:

- Body text.
- Metadata.
- Navigation labels.
- Buttons.
- Search interfaces.

Inter was selected because of its readability and versatility across multiple screen sizes and interface contexts.

3.6.3 Typography Hierarchy

The application maintains a structured typography scale to create predictable hierarchy and visual consistency.

Typical sizes include:

- 14 pt for metadata and secondary text.
- 16 pt for body text.
- 20 pt for section headings.
- 22 pt for feature headings.
- 30 pt for major titles.

This hierarchy improves scannability and helps users quickly distinguish between primary and secondary information.

3.7 Information Architecture

The information architecture of the application is designed to reduce cognitive load and improve discoverability.

Rather than presenting all functionality simultaneously, the system organizes content according to visitor intent and exploration workflow.

The main sections include:

- Home and recommendations.
- Artwork browsing.
- Search and discovery.
- QR-code scanning.
- Curated tours.
- Collections.
- Interactive map.
- User profile and saved content.

The navigation structure emphasizes recognition-based exploration, allowing visitors to identify available actions visually rather than relying on memorization.

Visual hierarchy is created through spacing, typography scale, alignment, and content grouping.

3.8 Navigation and Interaction Design

Navigation is designed to minimize interaction complexity during physical museum exploration.

The application uses familiar mobile interaction patterns so users can operate the system naturally without extensive learning.

The navigation system prioritizes:

- Fast access to core functions.
- Consistent interaction patterns.
- Minimal interaction depth.
- Large touch-friendly components.
- Clear feedback and transitions.

Frequently used features such as search, scanning, tours, and maps are placed prominently within the interface.

Left-aligned layouts are also used extensively because left alignment improves readability and scanning efficiency in mobile interfaces.

3.9 QR-Code Interaction Design

QR-code scanning is a major interaction mechanism within the system.

The scanning interface is designed to be:

- Visually minimal.
- Easy to access.
- Fast to operate.
- Understandable for first-time users.

The system also supports manual artwork number entry as a fallback mechanism in case scanning fails or camera permissions are unavailable.

This dual-access strategy improves reliability and accessibility.

3.10 Interactive Map Design

The interactive map is designed to function both as a navigation tool and as a contextual exploration interface.

Visitors can:

- View room layouts.
- Locate artworks.
- Identify nearby exhibits.
- Follow tour routes.
- Understand spatial relationships between galleries.

The map interface avoids excessive visual clutter and prioritizes readability and orientation clarity.

3.11 Accessibility and Inclusive Design

Accessibility considerations are integrated throughout the application rather than added as isolated features.

The system supports:

- Audio descriptions.
- Transcript support.
- Readable typography.
- High contrast interfaces.
- Dark mode.
- Responsive layouts.
- Multilingual presentation.

These features improve inclusiveness and support visitors with different sensory, linguistic, or cognitive needs.

3.12 Mockup Interfaces

Several interface Mockups were developed during the design process to evaluate interaction quality and visual consistency.

The Mockups include:

- Login interfaces.

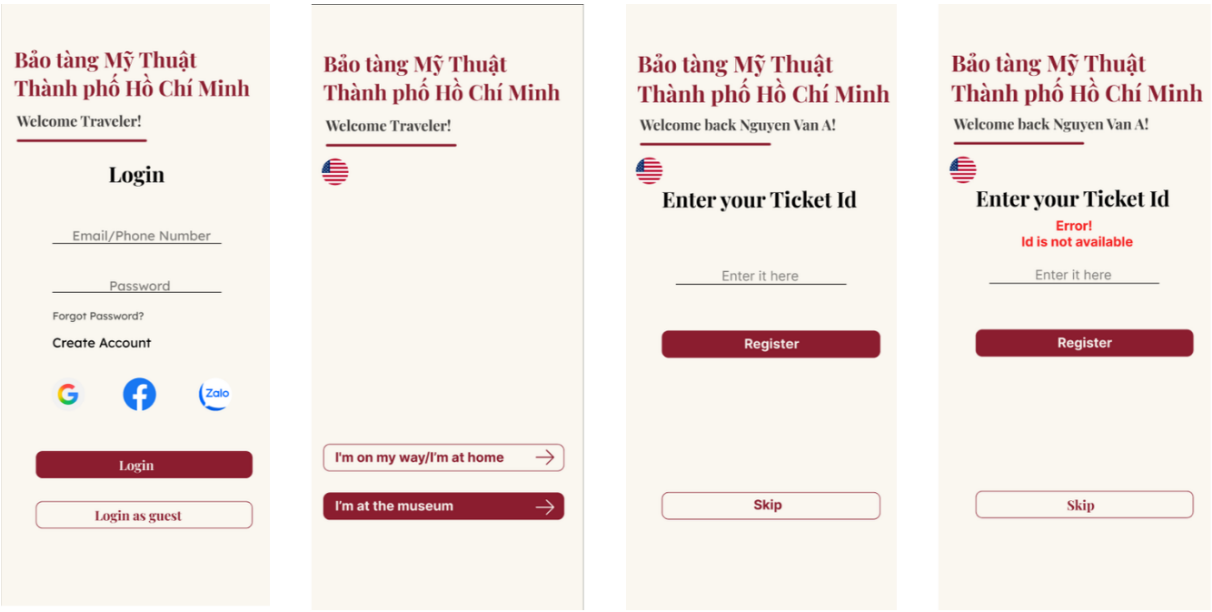


Figure 3.3: Login

- Search interfaces.

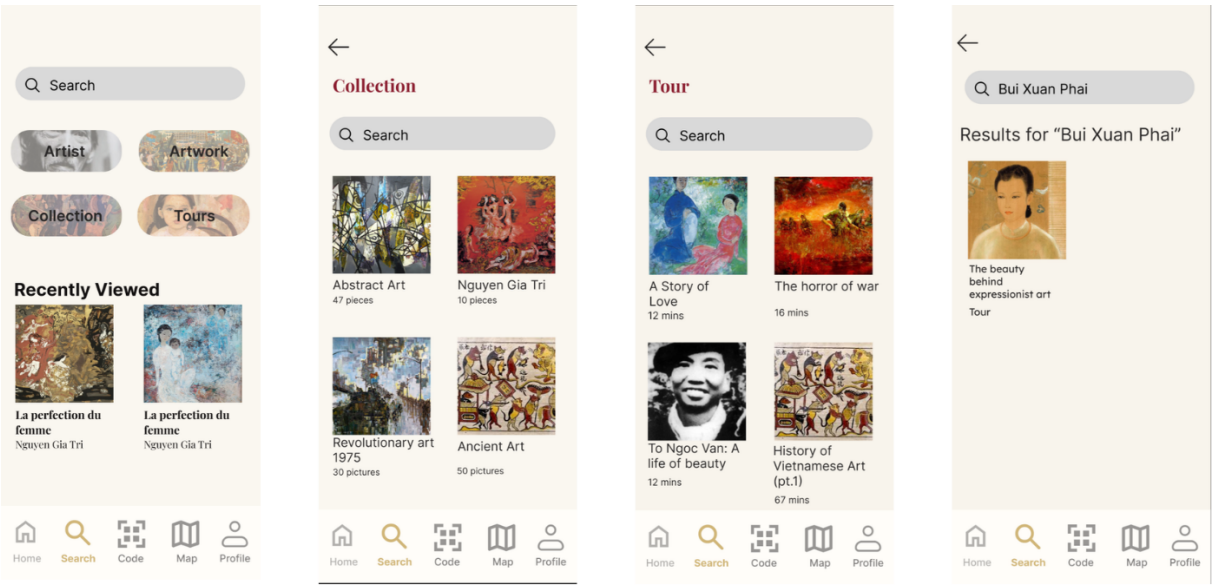


Figure 3.4: Search

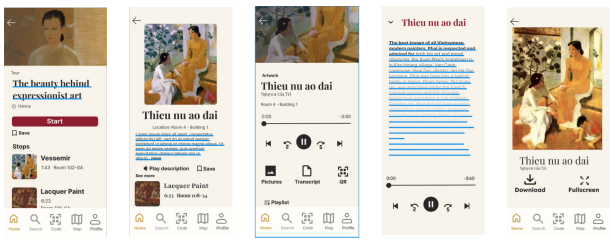


Figure 3.5: Tour and Artwork

- Artwork detail pages and Tours.
- QR-code scanning screens.

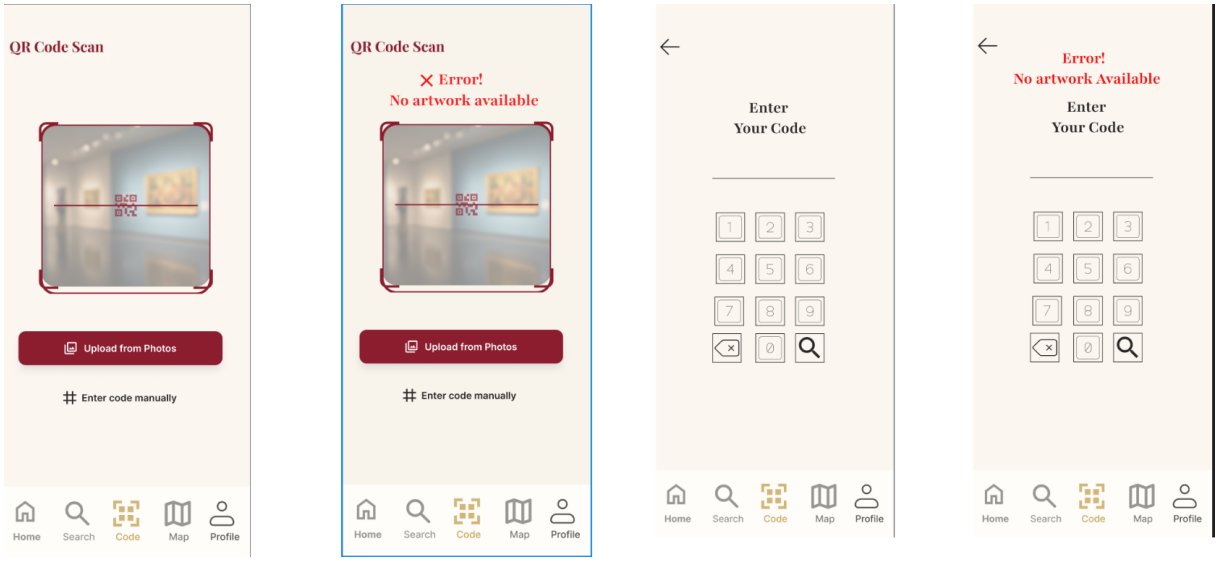


Figure 3.6: QR Code

- Dark mode variants.

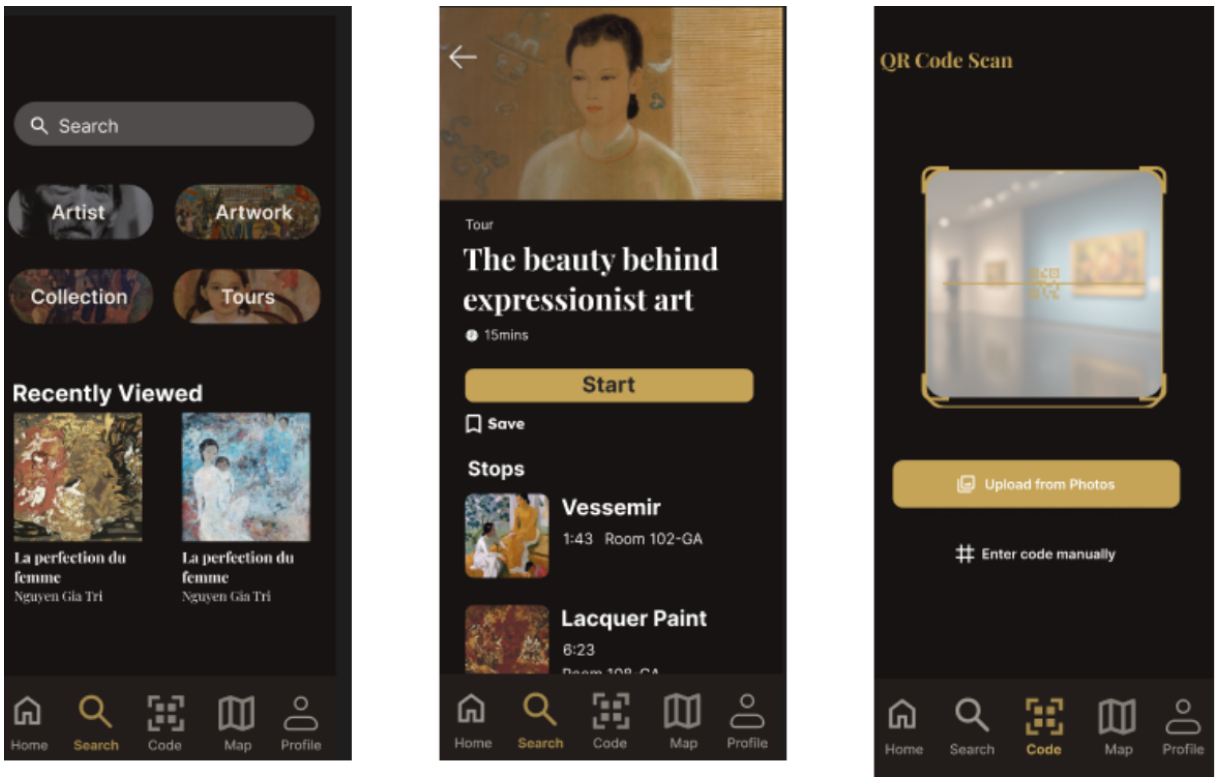


Figure 3.7: Dark Mode

- Vietnamese localization.

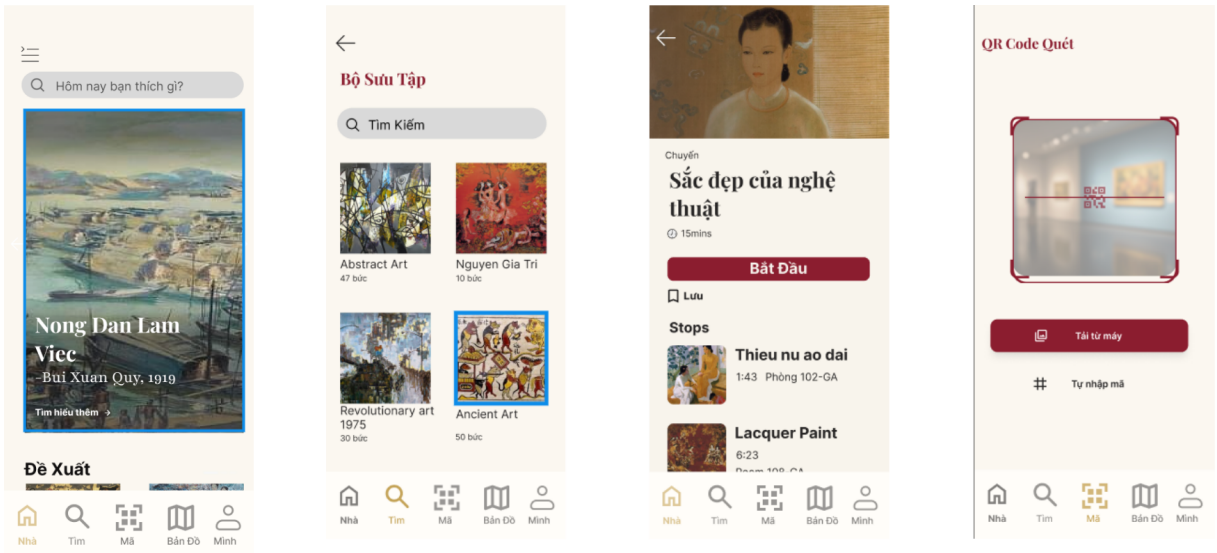


Figure 3.8: Vietnamese

- Tablet and iPad layouts.

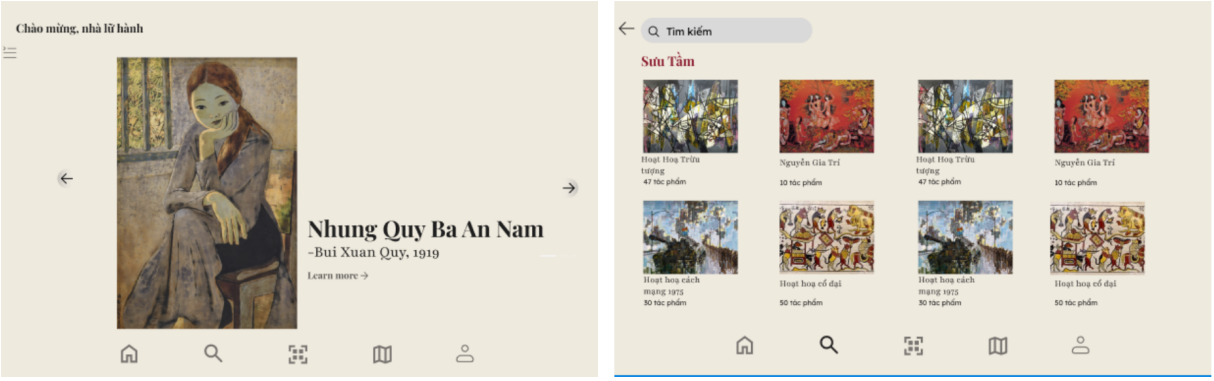


Figure 3.9: Table UI

These mockups were used to validate navigation structure, typography hierarchy, spacing consistency, and multimedia presentation.

3.13 Summary

In summary, the UI-UX design of the Museum Companion System aims to create an immersive, accessible, and aesthetically coherent digital museum experience.

The application combines usability-focused interaction design, museum-inspired visual identity, inclusive accessibility support, structured information architecture, and multimedia presentation into a unified interface system.

By balancing elegance, simplicity, readability, and intelligent interaction, the system attempts to function not merely as a digital information viewer, but as a true virtual companion that enhances cultural exploration and visitor engagement throughout the museum journey.

Chapter 4

System Design

4.1 Sequence Diagrams

Sequence diagrams are used to model the dynamic behavior of the system by illustrating the interaction flow between actors and system components over time. Unlike use-case diagrams, which describe functional relationships at a high level, sequence diagrams focus on the chronological order of messages exchanged between the user, the mobile application, and backend services during specific operations.

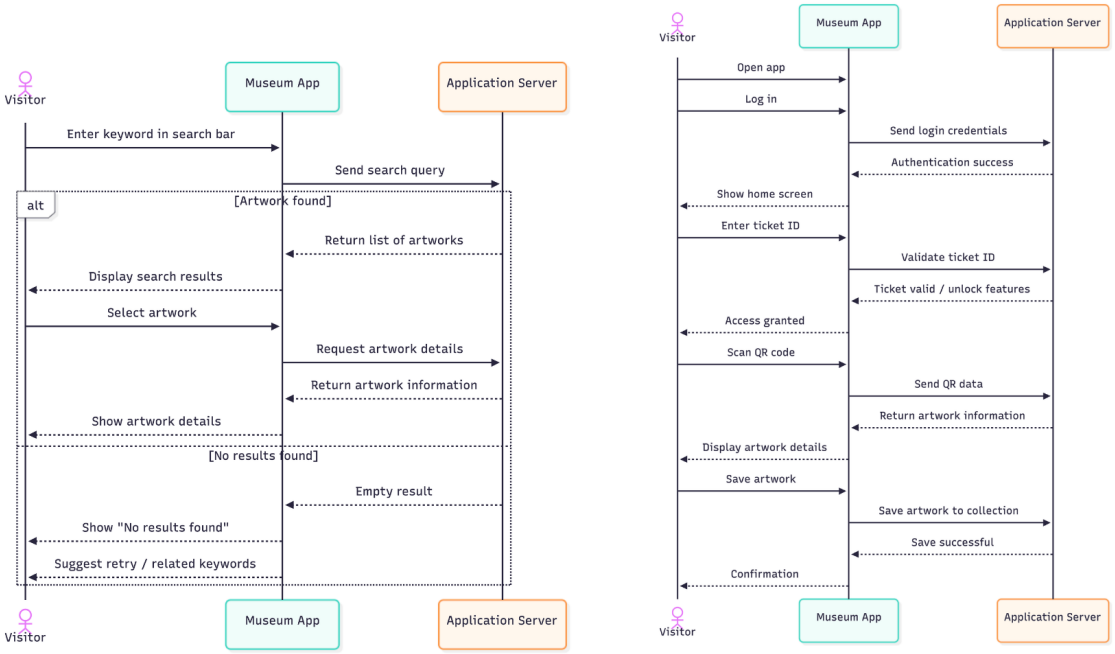


Figure 4.1: Sequence Diagrams

The sequence diagrams presented in this project describe two important interaction flows within the Museum Companion System:

- Artwork search and retrieval flow.
- Authentication, QR-code interaction, and artwork saving flow.

These diagrams help clarify how requests are processed internally and how the frontend application communicates with backend services during user interaction.

4.1.1 Artwork Search and Retrieval Flow

The first sequence diagram models the process of searching for artworks within the application and retrieving artwork details from the backend server.

The interaction begins when the *Visitor* enters a keyword into the search interface of the Museum App. The keyword may represent an artwork title, artist name, category, motif, or other searchable metadata.

After receiving the input, the Museum App sends a search query request to the Application Server. The server processes the query against the artwork database and returns the corresponding result set.

The sequence diagram models two possible outcomes using an alternative (`alt`) fragment:

- Artwork found.
- No matching result found.

Successful Search Flow

When matching artworks exist, the Application Server returns a list of artwork records to the Museum App. The application then displays the search results to the visitor.

After reviewing the result list, the visitor selects a specific artwork. The Museum App subsequently sends a request for detailed artwork information to the Application Server.

The server retrieves the corresponding artwork record, including metadata, description, multimedia references, and associated information, then returns the artwork details to the application.

Finally, the Museum App displays the artwork detail page to the visitor.

This interaction flow demonstrates several important architectural characteristics:

- The frontend application is responsible for interaction handling and presentation logic.
- The backend server is responsible for data retrieval and query processing.
- Search functionality is implemented as a request-response interaction between client and server.
- Detailed artwork information is retrieved only when required, reducing unnecessary data transfer.

The sequence also reflects the separation of concerns between interface behavior and backend data management.

No-Result Search Flow

If the Application Server cannot find any artwork matching the provided keyword, it returns an empty result set to the Museum App.

The application then displays a “No results found” message to the visitor and may additionally suggest retry actions or related keywords.

This alternative flow is important from both usability and reliability perspectives because it ensures graceful handling of unsuccessful searches rather than leaving the interface in an undefined state.

From a requirements perspective, this interaction supports the following functional behaviors:

- Keyword-based artwork search.
- Structured result retrieval.
- Error and empty-state handling.

- Dynamic retrieval of artwork details.

4.1.2 Authentication and QR-Code Interaction Flow

The second sequence diagram models a more complete interaction scenario involving authentication, QR-code scanning, artwork retrieval, and saving artworks to a personal collection.

The interaction begins when the visitor opens the Museum App and logs into the system.

Authentication Process

After the visitor enters login information, the Museum App sends authentication credentials to the Application Server.

The Application Server validates the credentials and returns an authentication success response if verification succeeds. Once authentication is completed, the application displays the home screen and grants access to authenticated features.

The visitor may then enter a ticket identifier. The Museum App forwards this ticket ID to the Application Server for validation.

If the ticket is valid, the server returns a success response and unlocks additional application functionality. The application subsequently grants access to the visitor.

This authentication flow demonstrates several technical aspects of the system:

- Authentication is handled through centralized backend validation.
- Access control logic is separated from frontend presentation logic.
- Feature availability depends on server-side authorization and validation.
- The application supports conditional access based on ticket verification.

QR-Code Artwork Access

After authentication and ticket validation, the visitor scans a QR code associated with a physical museum exhibit.

The Museum App extracts the QR-code data and sends the encoded artwork identifier to the Application Server.

The server retrieves the corresponding artwork information and returns the artwork record to the application. The Museum App then displays the artwork details to the visitor.

This interaction demonstrates how the system integrates physical museum exhibits with digital interpretation mechanisms.

From a system-design perspective, the QR-code interaction provides several advantages:

- Fast artwork retrieval with minimal user input.
- Reduced navigation complexity inside the application.
- Strong linkage between physical and digital museum environments.
- Improved contextual access to exhibit information.

Saving Artwork to Personal Collection

After viewing the artwork details, the visitor may choose to save the artwork into a personal collection or favorites list.

The Museum App sends a save request to the Application Server, which stores the artwork reference in the visitor's saved collection data.

After successful storage, the server returns a confirmation response to the application, and the Museum App displays a success confirmation to the visitor.

This interaction flow demonstrates the personalization capabilities of the system. It also illustrates how authenticated user-specific operations differ from public content retrieval operations.

From a requirements perspective, this flow supports:

- User authentication.
- Ticket validation.
- QR-code artwork retrieval.

- Personalized artwork saving.
- Persistent user-specific data management.

4.1.3 Architectural Interpretation

The sequence diagrams collectively illustrate several important architectural characteristics of the Museum Companion System.

First, the application follows a client-server interaction model where the Museum App acts as the presentation and interaction layer, while the Application Server performs validation, business logic processing, and data retrieval.

Second, the system uses request-response communication patterns for most operations, including search, authentication, artwork retrieval, and user personalization.

Third, the diagrams demonstrate clear separation between public visitor operations and authenticated user operations. General artwork browsing and searching may be accessible publicly, while saving artworks and personalized features require authentication and server-side identity management.

Fourth, the diagrams highlight the integration between physical museum interaction and digital functionality. QR-code scanning acts as a bridge between physical exhibits and digital content retrieval.

Finally, the diagrams reveal that the system supports both synchronous interaction flows and alternative execution paths, improving robustness and usability in real-world museum scenarios.

4.1.4 Summary

In summary, the sequence diagrams describe the dynamic interaction behavior of the Museum Companion System during search, authentication, QR-code access, and personalization operations.

The diagrams demonstrate how the frontend application, backend services, and visitors interact throughout the system lifecycle. They also clarify the chronological order of communication events, alternative interaction paths, and the responsibilities of each system component.

These interaction models provide an important foundation for later stages of implementation, API design, backend service development, and frontend integration.

4.2 Relational Schema

The relational schema of the Museum Companion System defines the structured data model used by the backend application and database layer. The schema is designed to support museum content management, user personalization, conversational assistant interaction, curated tours, and recommendation-related functionality.

The database follows a relational design implemented using PostgreSQL and managed through Prisma ORM. Each entity represents a specific domain concept within the system, while relationships between entities model interactions such as artwork ownership, tour composition, user personalization, and assistant conversations.

Figure 4.2 presents the relational schema of the system.

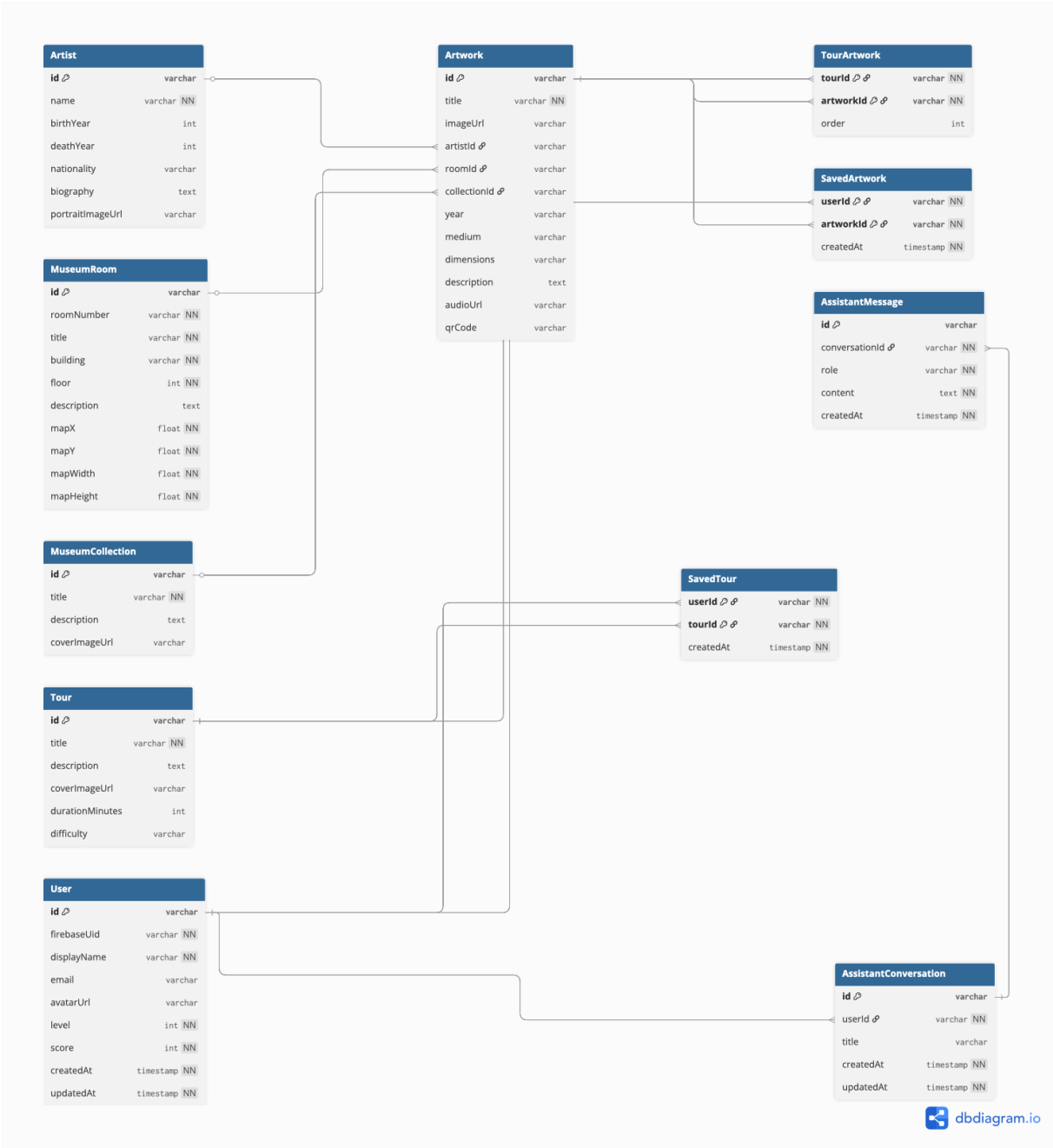


Figure 4.2: Relational schema of the Museum Companion System

4.2.1 Core Museum Entities

The core museum entities represent the primary cultural and spatial information stored in the system.

Artist

The `Artist` table stores information about artists whose artworks are displayed in the museum.

Its attributes include:

- `id`: unique identifier of the artist.
- `name`: full name of the artist.
- `birthYear`: artist birth year.
- `deathYear`: artist death year.
- `nationality`: artist nationality.
- `biography`: textual biography and historical information.
- `portraitImageUrl`: optional portrait image reference.

Each artist may be associated with multiple artworks, forming a one-to-many relationship between `Artist` and `Artwork`.

MuseumRoom

The `MuseumRoom` table represents physical exhibition spaces inside the museum.

Its attributes include:

- `id`: unique room identifier.
- `roomNumber`: room number displayed in the museum.
- `title`: room title.
- `building`: building identifier.
- `floor`: floor number.
- `description`: room description.
- `mapX`: horizontal coordinate on the digital map.
- `mapY`: vertical coordinate on the digital map.
- `mapWidth`: room width on the digital map.

- `mapHeight`: room height on the digital map.

The map-related attributes allow the frontend application to render rooms spatially inside the interactive museum map.

Each room may contain multiple artworks.

MuseumCollection

The `MuseumCollection` table stores thematic or curated artwork collections.

Its attributes include:

- `id`: collection identifier.
- `title`: collection title.
- `description`: collection description.
- `coverImageUrl`: optional collection cover image.

Collections group related artworks according to themes, artistic movements, historical periods, or curatorial categories.

One collection may contain multiple artworks.

Artwork

The `Artwork` table is the central content entity of the system.

It stores all major information related to museum exhibits.

Its attributes include:

- `id`: unique artwork identifier.
- `title`: artwork title.
- `imageUrl`: artwork image reference.
- `artistId`: foreign key referencing `Artist`.
- `roomId`: foreign key referencing `MuseumRoom`.
- `collectionId`: foreign key referencing `MuseumCollection`.

- `year`: artwork creation year.
- `medium`: artistic medium.
- `dimensions`: artwork dimensions.
- `description`: textual artwork interpretation.
- `audioUrl`: audio narration reference.
- `qrCode`: QR-code identifier associated with the physical exhibit.

The `Artwork` entity serves as the central connection point for multiple relationships including artists, rooms, collections, tours, recommendations, and user personalization.

4.2.2 Tour and Exploration Entities

The system supports curated exploration experiences through tours and tour-artwork relationships.

Tour

The `Tour` table stores curated museum tours.

Its attributes include:

- `id`: unique tour identifier.
- `title`: tour title.
- `description`: tour description.
- `coverImageUrl`: optional tour cover image.
- `durationMinutes`: estimated duration.
- `difficulty`: difficulty or complexity classification.

Tours provide structured museum exploration experiences based on themes, artistic styles, or historical narratives.

TourArtwork

The `TourArtwork` table is a junction table implementing the many-to-many relationship between tours and artworks.

Its attributes include:

- `tourId`: foreign key referencing `Tour`.
- `artworkId`: foreign key referencing `Artwork`.
- `order`: ordering position inside the tour.

This design allows a single artwork to appear in multiple tours while preserving custom ordering for each tour sequence.

The inclusion of the `order` attribute is important because curated tours require explicit navigation ordering rather than unordered membership.

4.2.3 User and Personalization Entities

The system supports personalization and user-specific interactions through user-related entities.

User

The `User` table stores application-level user profiles.

Authentication identity is managed externally through Firebase Authentication, while the backend stores personalized application data.

Its attributes include:

- `id`: unique internal user identifier.
- `firebaseUid`: Firebase authentication identifier.
- `displayName`: user display name.
- `email`: user email address.
- `avatarUrl`: optional avatar image.

- `level`: gamification or progression level.
- `score`: accumulated user score.
- `createdAt`: account creation timestamp.
- `updatedAt`: profile update timestamp.

The separation between Firebase identity and application-specific profile data improves flexibility and security.

SavedArtwork

The `SavedArtwork` table stores user-bookmarked artworks.

It implements a many-to-many relationship between users and artworks.

Its attributes include:

- `userId`: foreign key referencing `User`.
- `artworkId`: foreign key referencing `Artwork`.
- `createdAt`: save timestamp.

This table enables personalization features such as favorites, saved collections, and revisit functionality.

SavedTour

The `SavedTour` table stores tours bookmarked by users.

Its attributes include:

- `userId`: foreign key referencing `User`.
- `tourId`: foreign key referencing `Tour`.
- `createdAt`: save timestamp.

This supports personalized exploration and future revisit functionality.

4.2.4 Assistant and Conversational Entities

The intelligent assistant system maintains persistent conversation history through conversational entities.

AssistantConversation

The `AssistantConversation` table stores conversation sessions between users and the assistant.

Its attributes include:

- `id`: conversation identifier.
- `userId`: foreign key referencing `User`.
- `title`: conversation title.
- `createdAt`: creation timestamp.
- `updatedAt`: last update timestamp.

One user may own multiple conversations.

AssistantMessage

The `AssistantMessage` table stores individual messages belonging to conversations.

Its attributes include:

- `id`: message identifier.
- `conversationId`: foreign key referencing `AssistantConversation`.
- `role`: message role.
- `content`: message content.
- `createdAt`: message timestamp.

The `role` field distinguishes between user messages and assistant-generated responses.

This schema supports persistent conversational memory and allows visitors to revisit previous assistant interactions.

4.2.5 Relationship Analysis

The relational schema contains several important relationships.

One-to-Many Relationships

The following one-to-many relationships exist:

- One artist may create many artworks.
- One museum room may contain many artworks.
- One collection may contain many artworks.
- One user may own many assistant conversations.
- One conversation may contain many assistant messages.

These relationships model hierarchical ownership and organizational structures.

Many-to-Many Relationships

The system also includes multiple many-to-many relationships implemented through junction tables.

- Tours and artworks are connected through `TourArtwork`.
- Users and artworks are connected through `SavedArtwork`.
- Users and tours are connected through `SavedTour`.

This design improves flexibility and supports personalization functionality.

4.2.6 Schema Design Considerations

Several important design decisions influenced the schema structure.

Normalization

The schema follows relational normalization principles to reduce redundancy and maintain consistency.

Museum entities such as artists, rooms, collections, and artworks are separated into dedicated tables rather than duplicated across records.

Extensibility

The schema is designed to support future feature expansion.

Possible future extensions include:

- Recommendation tracking.
- User analytics.
- Achievement systems.
- Augmented reality metadata.
- Indoor positioning data.
- Multilingual content storage.

Support for Intelligent Features

The schema is structured to support Retrieval-Augmented Generation and recommendation systems.

Artwork descriptions, metadata, and relationships provide rich contextual information that can later be embedded into vector representations for semantic retrieval.

Spatial Awareness Support

The inclusion of map coordinates in the `MuseumRoom` entity supports spatial rendering and navigation functionality inside the mobile application.

4.2.7 Summary

The relational schema provides the structured data foundation of the Museum Companion System.

The schema models museum content, tours, personalization, spatial information, and conversational interaction using a normalized relational structure implemented in PostgreSQL.

By combining core museum entities, user personalization tables, and conversational assistant entities, the schema supports both traditional museum application functionality and advanced intelligent interaction features such as recommendations, Retrieval-Augmented Generation, and smart journey generation.

4.3 Class Diagram

The class diagram represents the main backend structure of the Museum Companion System. It describes the relationship between controllers, services, infrastructure components, authentication guards, data transfer objects, response models, and domain entities.

The system follows a layered architecture. Controllers receive requests from the mobile application, services contain the main business logic, infrastructure components communicate with external systems and the database, and entities represent the persistent data model of the application.

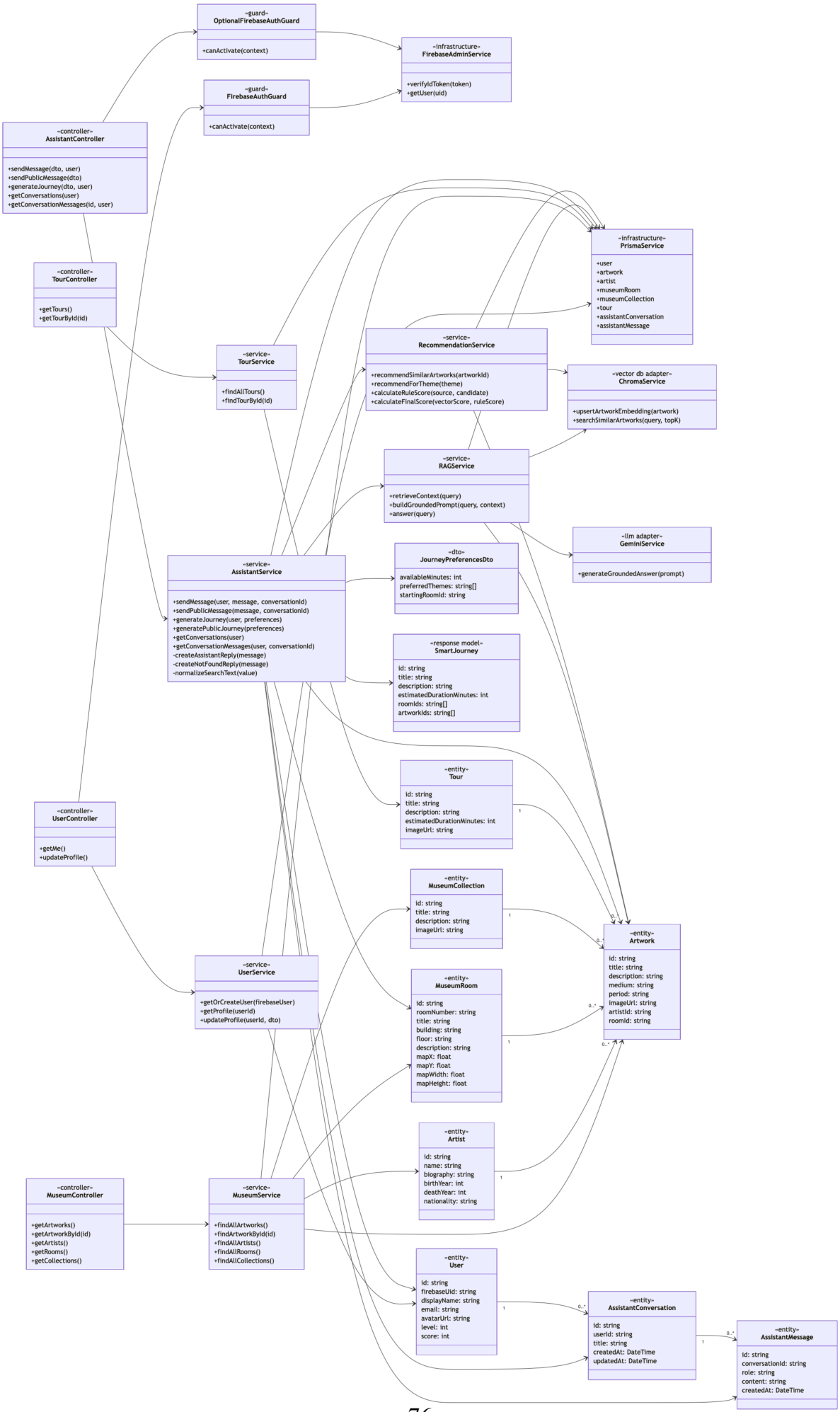


Figure 4.3: Class diagram of the Museum Companion System

4.3.1 Controller Layer

The controller layer is responsible for exposing backend functions through API endpoints. Controllers do not directly contain complex business logic. Instead, they receive requests, validate input when necessary, and delegate processing to the corresponding service classes.

MuseumController

The `MuseumController` handles public museum content retrieval. It provides endpoints for retrieving artworks, artists, museum rooms, and collections. This controller is mainly used by the mobile application when users browse museum content, open artwork detail pages, or access collection-based views.

The main operations include:

- `getArtworks()` for retrieving artwork lists.
- `getArtworkById(id)` for retrieving a specific artwork.
- `getArtists()` for retrieving artist information.
- `getRooms()` for retrieving museum room data.
- `getCollections()` for retrieving museum collections.

TourController

The `TourController` manages API endpoints related to curated museum tours. It allows the client application to retrieve all available tours or retrieve the details of a specific tour.

The main operations include:

- `getTours()` for retrieving all tours.
- `getTourById(id)` for retrieving one selected tour.

UserController

The `UserController` handles authenticated user profile operations. It is used when a signed-in visitor accesses or updates their personal profile.

The main operations include:

- `getMe()` for retrieving the current authenticated user profile.
- `updateProfile()` for updating user profile information.

AssistantController

The `AssistantController` exposes endpoints related to the intelligent assistant and smart journey generation. It supports both authenticated and public assistant interaction.

The main operations include:

- `sendMessage(dto, user)` for sending an authenticated assistant message.
- `sendPublicMessage(dto)` for sending a public assistant message.
- `generateJourney(dto, user)` for generating a personalized museum journey.
- `getConversations(user)` for retrieving conversation history.
- `getConversationMessages(id, user)` for retrieving messages inside a conversation.

4.3.2 Authentication and Guard Layer

Authentication and authorization are handled through guard classes and Firebase integration. This separates access control from controller logic and ensures that protected endpoints can only be accessed by valid users.

FirebaseAuthGuard

The `FirebaseAuthGuard` protects endpoints that require authentication. It validates the request context before allowing access to controller methods.

Its main operation is:

- `canActivate(context)` for checking whether the request contains a valid Firebase identity token.

OptionalFirebaseAuthGuard

The `OptionalFirebaseAuthGuard` supports endpoints that can be accessed by both guest users and authenticated users. This is useful for public assistant features, public recommendations, or features where personalization is optional.

Its main operation is:

- `canActivate(context)` for checking authentication when present while still allowing guest access.

FirebaseAdminService

The `FirebaseAdminService` acts as the infrastructure service responsible for communicating with Firebase Authentication. It verifies identity tokens and retrieves Firebase user records.

The main operations include:

- `verifyIdToken(token)` for validating Firebase tokens.
- `getUser(uid)` for retrieving Firebase user information.

4.3.3 Service Layer

The service layer contains the main business logic of the system. Services coordinate database operations, assistant behavior, recommendation logic, user profile management, and tour retrieval.

MuseumService

The `MuseumService` provides access to core museum data. It retrieves artworks, artists, rooms, and collections from the database through the persistence layer.

The main operations include:

- `findAllArtworks()`.

- `findArtworkById(id)`.
- `findAllArtists()`.
- `findAllRooms()`.
- `findAllCollections()`.

TourService

The `TourService` handles curated tour retrieval. It supports tour listing and tour detail access.

The main operations include:

- `findAllTours()`.
- `findTourById(id)`.

UserService

The `UserService` manages application-level user profiles. While Firebase handles authentication identity, the backend stores application-specific user data such as display name, level, score, saved content, and user preferences.

The main operations include:

- `getOrCreateUser(firebaseUser)` for creating or retrieving a backend user profile linked to a Firebase account.
- `getProfile(userId)` for retrieving profile data.
- `updateProfile(userId, dto)` for updating profile data.

AssistantService

The `AssistantService` is responsible for assistant interaction, conversation management, and smart journey generation. It coordinates the retrieval-augmented generation process, stores conversation messages, and generates personalized journey outputs.

The main operations include:

- `sendMessage(user, message, conversationId)`.
- `sendPublicMessage(message, conversationId)`.
- `generateJourney(user, preferences)`.
- `generatePublicJourney(preferences)`.
- `getConversations(user)`.
- `getConversationMessages(user, conversationId)`.
- `createAssistantReply(message)`.
- `createUserReply(message)`.
- `normalizeSearchText(value)`.

RecommendationService

The `RecommendationService` generates artwork recommendations by combining semantic similarity and rule-based scoring. This allows the system to recommend artworks using both vector similarity and structured museum metadata.

The main operations include:

- `recommendSimilarArtworks(artworkId)`.
- `recommendForTheme(theme)`.
- `calculateRuleScore(score, candidate)`.
- `calculateFinalScore(vectorScore, ruleScore)`.

This design supports explainable recommendation behavior because recommendations can be based on visible factors such as artist similarity, collection similarity, artwork categories, tags, and semantic similarity.

RAGService

The `RAGService` implements the retrieval-augmented generation pipeline used by the assistant. It retrieves relevant museum context, builds a grounded prompt, and generates an answer through the language model adapter.

The main operations include:

- `retrieveContext(query)` for retrieving relevant museum knowledge.
- `buildGroundedPrompt(query, context)` for constructing a prompt grounded in museum data.
- `answer(query)` for returning a generated assistant answer.

4.3.4 Infrastructure and Adapter Layer

The infrastructure layer provides access to external technologies such as the database, vector database, Firebase, and language model service. These components are separated from business logic to improve maintainability and replaceability.

PrismaService

The `PrismaService` is the database access component of the system. It communicates with the PostgreSQL database through Prisma ORM and exposes access to persistent models such as users, artworks, artists, rooms, collections, tours, assistant conversations, and assistant messages.

ChromaService

The `ChromaService` acts as the vector database adapter. It stores and searches artwork embeddings for semantic retrieval and recommendation features.

The main operations include:

- `upsertArtworkEmbedding(artwork)` for storing or updating artwork embeddings.
- `searchSimilarArtworks(query, topK)` for retrieving semantically similar artworks.

GeminiService

The `GeminiService` is the language model adapter used by the intelligent assistant. It receives grounded prompts from the RAG service and returns generated responses.

The main operation is:

- `generateGroundedAnswer(prompt)`.

4.3.5 DTOs and Response Models

DTOs and response models define structured input and output objects used by the service layer.

JourneyPreferencesDto

The `JourneyPreferencesDto` represents the visitor's input preferences for smart journey generation.

Its fields include:

- `availableMinutes`: the amount of time the visitor has.
- `preferredThemes`: a list of preferred themes.
- `startingRoomId`: the starting room of the visitor.

SmartJourney

The `SmartJourney` response model represents a generated museum journey.

Its fields include:

- `id`.
- `title`.
- `description`.
- `estimatedDurationMinutes`.
- `rooms`.
- `artworkIds`.

4.3.6 Domain Entity Layer

The entity layer represents the core persistent concepts of the Museum Companion System.

Artwork

The `Artwork` entity is the central content entity of the system. It represents an artwork displayed in the museum.

Its main attributes include:

- `id`.
- `title`.
- `description`.
- `medium`.
- `period`.
- `imageUrl`.
- `artistId`.
- `roomId`.

Each artwork may be associated with an artist, a museum room, a collection, and one or more tours.

Artist

The `Artist` entity stores information about artwork creators.

Its main attributes include:

- `id`.
- `name`.
- `biography`.

- `birthYear`.
- `deathYear`.
- `nationality`.

An artist may be linked to multiple artworks.

MuseumRoom

The `MuseumRoom` entity represents a physical room or exhibition space inside the museum.

Its main attributes include:

- `id`.
- `roomNumber`.
- `title`.
- `building`.
- `floor`.
- `description`.
- `mapX`.
- `mapY`.
- `mapWidth`.
- `mapHeight`.

The map-related attributes allow the mobile application to place rooms and artworks on the digital museum map.

MuseumCollection

The `MuseumCollection` entity represents a group of related artworks.

Its main attributes include:

- `id`.
- `title`.
- `description`.
- `imageUrl`.

Collections support thematic browsing and structured exploration of museum content.

Tour

The `Tour` entity represents a curated museum tour.

Its main attributes include:

- `id`.
- `title`.
- `description`.
- `estimatedDurationMinutes`.
- `imageUrl`.

Tours are connected to artworks and provide guided exploration paths for visitors.

User

The `User` entity represents an application user linked to Firebase Authentication.

Its main attributes include:

- `id`.
- `firebaseUid`.

- displayName.
- email.
- avatarUrl.
- level.
- score.

This entity stores application-specific profile information while authentication identity is managed externally by Firebase.

AssistantConversation

The AssistantConversation entity represents a conversation session between a user and the intelligent assistant.

Its main attributes include:

- id.
- userId.
- title.
- createdAt.
- updatedAt.

A user may have multiple assistant conversations.

AssistantMessage

The AssistantMessage entity represents an individual message inside an assistant conversation.

Its main attributes include:

- id.
- conversationId.

- `role`.
- `content`.
- `createdAt`.

The `role` field distinguishes between user messages and assistant responses.

4.3.7 Architectural Interpretation

The class diagram shows that the system follows a clear separation of concerns. Controllers handle request routing, services implement business logic, infrastructure components manage external dependencies, and entities represent persistent domain data.

The diagram also shows that artificial intelligence features are isolated from the rest of the system through dedicated services. The `RAGService`, `ChromaService`, and `GeminiService` separate retrieval, vector search, and language generation responsibilities. This makes the assistant easier to maintain and reduces direct dependency between controllers and AI infrastructure.

The recommendation system is designed as a hybrid approach. It combines semantic similarity from vector search with rule-based scoring based on artwork metadata. This provides a more controllable and explainable recommendation mechanism.

Authentication is handled through Firebase guards and the Firebase administration service. This allows the system to rely on Firebase for identity verification while still maintaining internal user profiles and personalization data in the backend database.

Overall, the class diagram demonstrates a modular backend structure suitable for a museum companion application. It supports public content browsing, authenticated personalization, curated tours, AI-assisted exploration, recommendation logic, and administrative extensibility.

Chapter 5

System Architecture

5.1 Architectural Overview

The Museum Companion System is designed using a layered client-server architecture that separates presentation logic, business logic, persistence, authentication, and intelligent retrieval components into independent modules. The architecture is intended to support maintainability, scalability, extensibility, and clear separation of concerns while remaining practical for implementation and future expansion.

At a high level, the system consists of the following major components:

- iOS mobile frontend application.
- Backend API server.
- Relational database system.
- Vector database for semantic retrieval.
- Authentication infrastructure.
- Retrieval-Augmented Generation (RAG) pipeline.

The frontend application is implemented using SwiftUI and SwiftData and acts as the primary interaction layer between visitors and museum content. The backend is implemented using NestJS and follows a modular monolith architecture. PostgreSQL is

used for structured persistent data, while ChromaDB stores vector embeddings used for semantic search and intelligent assistant functionality.

The system also integrates Firebase Authentication for identity management and Gemini 2.5 Flash for grounded response generation.

Figure 5.1 illustrates the high-level architecture of the system.

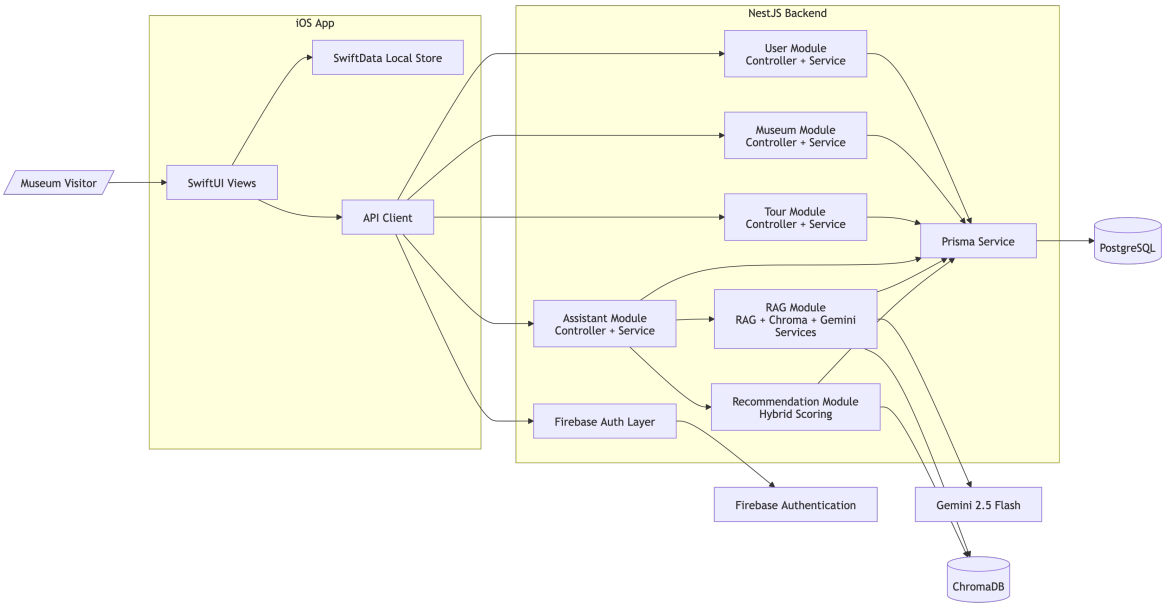


Figure 5.1: High-level architecture of the Museum Companion System

5.2 High-Level System Components

5.2.1 iOS Mobile Application

The iOS mobile application functions as the primary user-facing component of the system. It provides the graphical user interface through which museum visitors interact with artworks, tours, collections, maps, and assistant features.

The mobile frontend is implemented using:

- SwiftUI for interface construction.
- SwiftData for local persistence and caching.
- REST API communication with the backend server.

The application is responsible for:

- Artwork browsing.
- QR-code interaction.
- Search and filtering.
- Tour navigation.
- Interactive map visualization.
- Assistant interaction.
- Offline caching of previously loaded content.
- User personalization and saved content.

The frontend does not directly access the database or external AI services. All business logic and secure operations are delegated to the backend API.

5.2.2 Backend API Server

The backend API server is implemented using NestJS and acts as the central orchestration layer of the system.

Its responsibilities include:

- Processing client requests.
- Managing authentication and authorization.
- Retrieving museum data.
- Handling assistant conversations.
- Generating recommendations.
- Managing tours and collections.
- Coordinating Retrieval-Augmented Generation workflows.
- Communicating with the database and vector database.

The backend exposes RESTful API endpoints consumed by the iOS application.

5.2.3 PostgreSQL Database

The PostgreSQL database stores structured application data.

This includes:

- Artworks.
- Artists.
- Museum rooms.
- Collections.
- Tours.
- User profiles.
- Saved artworks.
- Saved tours.
- Assistant conversations.
- Assistant messages.

Prisma ORM is used as the database abstraction layer for interacting with PostgreSQL.

5.2.4 Vector Database

The system uses ChromaDB as a vector database for semantic retrieval and recommendation features.

The vector database stores artwork embeddings generated from museum content. These embeddings enable semantic similarity search and contextual retrieval for the intelligent assistant.

The vector database is primarily used for:

- Semantic artwork retrieval.
- Similar artwork recommendation.

- Retrieval-Augmented Generation context retrieval.
- Intelligent search functionality.

5.2.5 Firebase Authentication

Firebase Authentication is used as the external identity provider of the system.

Firebase handles:

- User login.
- Identity verification.
- Token generation.
- Session authentication.

The backend validates Firebase identity tokens before allowing access to protected operations such as saving artworks, managing conversations, or accessing personalization features.

5.2.6 Gemini 2.5 Flash Integration

The system integrates Gemini 2.5 Flash as the language model used by the intelligent assistant.

The language model is not used directly without contextual retrieval. Instead, the backend first retrieves relevant museum information from ChromaDB and structured museum data before constructing grounded prompts.

This architecture reduces hallucination risk and ensures that generated responses remain tied to verified museum content.

5.3 Detailed Backend Architecture

Figure 5.2 presents the detailed internal architecture of the backend system.

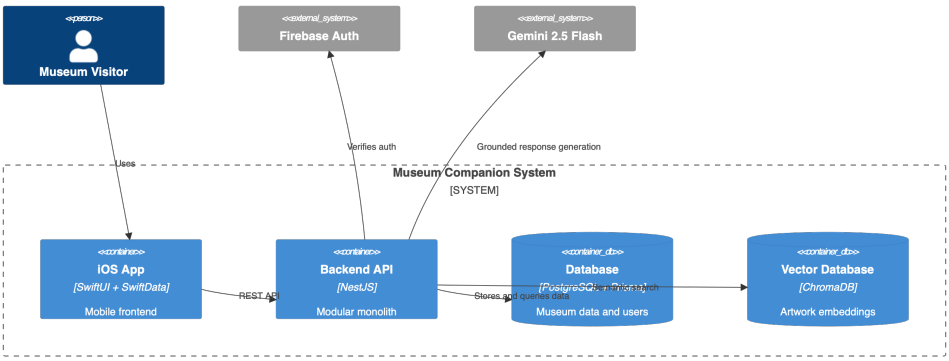


Figure 5.2: Detailed backend and frontend architecture

The backend is divided into multiple functional modules.

5.3.1 Museum Module

The Museum Module manages public museum content including artworks, artists, rooms, and collections.

Its responsibilities include:

- Artwork retrieval.
- Artist retrieval.
- Collection retrieval.
- Room retrieval.
- Museum content organization.

5.3.2 Tour Module

The Tour Module handles curated museum tours.

Its responsibilities include:

- Tour retrieval.

- Tour detail access.
- Tour-artwork relationships.
- Guided exploration support.

5.3.3 Assistant Module

The Assistant Module manages conversational interaction between visitors and the intelligent assistant.

Its responsibilities include:

- Conversation management.
- Assistant messaging.
- Smart journey generation.
- Personalized assistant interaction.

5.3.4 RAG Module

The Retrieval-Augmented Generation (RAG) Module coordinates semantic retrieval and grounded response generation.

It integrates:

- ChromaDB vector retrieval.
- Museum context retrieval.
- Prompt construction.
- Gemini response generation.

The RAG pipeline prevents the assistant from relying purely on general language generation and instead grounds responses in museum-specific data.

5.3.5 Recommendation Module

The Recommendation Module implements hybrid recommendation logic.

It combines:

- Vector similarity scoring.
- Rule-based scoring.
- Metadata similarity.
- Theme-based recommendation.

This hybrid approach improves recommendation quality and explainability compared to pure embedding similarity.

5.3.6 Firebase Authentication Layer

The Firebase Authentication Layer validates Firebase tokens and protects authenticated backend operations.

This layer separates identity verification from business logic and improves system security.

5.4 Frontend Architecture Using MVVM

The iOS mobile application follows the Model-View-ViewModel (MVVM) architectural pattern.

MVVM was selected because it improves separation of concerns, interface maintainability, and state management within SwiftUI applications.

5.4.1 View Layer

The View layer contains SwiftUI interface components.

Views are responsible only for presentation and user interaction handling.

Examples include:

- Artwork screens.
- Tour screens.
- Search interfaces.
- QR-code scanning interfaces.
- Map views.
- Assistant chat screens.

Views do not directly contain networking or persistence logic.

5.4.2 ViewModel Layer

The ViewModel layer acts as the mediator between the user interface and business logic.

ViewModels are responsible for:

- Managing screen state.
- Performing API requests.
- Processing user interaction.
- Preparing data for display.
- Managing loading and error states.

This layer improves testability and reduces coupling between interface components and backend communication.

5.4.3 Model Layer

The Model layer contains domain models and persistence objects.

SwiftData is used to store local application data such as:

- Cached artworks.
- Saved content.

- Conversation history.
- Offline-accessible data.

The model layer also mirrors backend entities to simplify frontend-backend synchronization.

5.4.4 Advantages of MVVM

The use of MVVM provides several architectural advantages:

- Clear separation between UI and logic.
- Better SwiftUI state management.
- Improved maintainability.
- Easier testing of business logic.
- Better scalability for future feature growth.

The architecture is particularly suitable for SwiftUI because SwiftUI relies heavily on reactive state updates.

5.5 Backend Architecture Using Modular Monolith

The backend follows a modular monolith architecture.

A modular monolith is a single deployable backend application internally divided into isolated functional modules.

Instead of deploying many distributed microservices, the system keeps all modules inside one backend application while preserving strong logical separation.

5.5.1 Reasons for Choosing a Modular Monolith

A modular monolith architecture was selected because it provides a balance between maintainability and implementation complexity.

Compared to microservices, the modular monolith approach:

- Reduces deployment complexity.
- Simplifies local development.
- Avoids distributed system overhead.
- Eliminates inter-service network latency.
- Simplifies database transactions.

At the same time, modular separation still allows the system to maintain clean boundaries between features.

5.5.2 Module Isolation

Each module encapsulates its own:

- Controllers.
- Services.
- DTOs.
- Business logic.

Examples include:

- Museum Module.
- Tour Module.
- Assistant Module.
- Recommendation Module.
- User Module.

This improves maintainability and allows future extraction into independent services if needed.

5.5.3 Advantages of the Modular Monolith Approach

The modular monolith architecture provides several important advantages:

- Strong separation of concerns.
- Easier debugging and deployment.
- Better development speed.
- Simpler infrastructure requirements.
- Easier coordination between AI and museum data modules.
- Reduced operational complexity compared to microservices.

The architecture is especially appropriate for medium-scale systems that require clear modularity but do not yet justify fully distributed deployment.

5.6 Architectural Characteristics

Several important architectural characteristics can be identified from the overall system design.

5.6.1 Layered Separation of Concerns

The system clearly separates:

- Presentation logic.
- Business logic.
- Persistence logic.
- Authentication.
- AI retrieval.
- External infrastructure integration.

This improves maintainability and extensibility.

5.6.2 AI Isolation

Artificial intelligence components are isolated into dedicated modules and services rather than directly embedded into controllers or frontend logic.

This makes the system easier to maintain and allows future replacement of AI providers without major architectural redesign.

5.6.3 Hybrid Recommendation Architecture

Recommendations are generated using both:

- Semantic vector similarity.
- Rule-based metadata scoring.

This hybrid approach improves explainability and recommendation quality.

5.6.4 Offline Support

SwiftData local persistence allows the application to cache previously loaded museum content and support offline interaction.

This improves usability inside museum environments where network quality may vary.

5.6.5 Scalability and Extensibility

The architecture allows future integration of features such as:

- Augmented reality.
- Indoor positioning systems.
- Gamification.
- Multilingual AI support.
- Advanced recommendation models.
- Analytics systems.

5.7 Summary

In summary, the Museum Companion System uses a layered client-server architecture combining an MVVM-based SwiftUI frontend with a modular monolith NestJS backend.

The architecture integrates structured persistence, semantic retrieval, authentication infrastructure, and Retrieval-Augmented Generation into a unified system suitable for intelligent museum exploration.

By separating frontend presentation, backend business logic, AI orchestration, and persistence infrastructure into dedicated layers and modules, the system achieves maintainability, scalability, extensibility, and support for future intelligent museum interaction features.

Chapter 6

Deployment

6.1 Technology Stack

The Museum Companion System is implemented using a combination of modern frontend, backend, database, authentication, and artificial intelligence technologies. The selected technology stack was chosen to support modular development, maintainability, scalability, and intelligent interaction features.

6.1.1 Frontend Technologies

The mobile frontend application is developed for iOS using SwiftUI and SwiftData.

- **SwiftUI** is used for building the user interface through a declarative and reactive programming model. It supports responsive layouts, smooth navigation, and maintainable UI development.
- **SwiftData** is used for local persistence and offline caching of museum content such as artworks, saved items, and conversation history.
- **AVFoundation** and native iOS frameworks are used for multimedia playback and QR-code interaction.

6.1.2 Backend Technologies

The backend server is implemented using NestJS and TypeScript.

- **NestJS** provides a modular server-side architecture suitable for building scalable REST APIs and structured backend modules.
- **TypeScript** improves maintainability and type safety throughout the backend codebase.
- **Prisma ORM** is used as the database abstraction layer for interacting with PostgreSQL.

6.1.3 Database Technologies

The system uses both relational and vector databases.

- **PostgreSQL** stores structured application data including artworks, artists, users, tours, collections, and assistant conversations.
- **ChromaDB** functions as the vector database used for semantic retrieval, artwork similarity search, and Retrieval-Augmented Generation workflows.

6.1.4 Authentication and Cloud Services

The system integrates Firebase Authentication for user identity management.

- **Firebase Authentication** handles user login, token generation, and identity verification.
- **Firebase Storage** may be used for storing multimedia assets such as artwork images and audio files.

6.1.5 Artificial Intelligence Technologies

The intelligent assistant and recommendation features rely on Retrieval-Augmented Generation and semantic retrieval technologies.

- **Gemini 2.5 Flash** is used as the large language model for grounded response generation.

- **Retrieval-Augmented Generation (RAG)** combines semantic retrieval with grounded prompt construction to improve factual correctness and reduce hallucination.
- **Embedding-based semantic search** is used for artwork similarity retrieval and recommendation generation.

6.1.6 Development and Deployment Tools

Several supporting tools are used during development and testing.

- **Xcode** is used for iOS application development.
- **Postman** is used for API testing and endpoint validation.
- **Docker** may be used for containerized backend deployment and database management.
- **Git and GitHub** are used for version control and collaborative development.

The selected technology stack provides a balance between modern frontend development, scalable backend architecture, intelligent retrieval capabilities, and maintainable system design.

6.2 Deployment

The Museum Companion System is designed using a client-server deployment model. The iOS mobile application is deployed to user devices and communicates with the backend server through RESTful API requests over the internet.

The backend API is deployed as a centralized server application and is responsible for business logic processing, authentication, recommendation generation, assistant interaction, and database communication.

PostgreSQL is deployed as the primary relational database for structured museum data, while ChromaDB is deployed separately to support semantic retrieval and vector-based search operations.

Firebase Authentication is used as an external cloud-based identity provider for user authentication and token verification.

The deployment architecture allows the frontend, backend, database, and AI-related services to operate independently while maintaining secure and scalable communication between components.

6.3 Testing Plans

Testing is an important phase of the Museum Companion System development process. The objective of testing is to ensure that the application satisfies functional requirements, maintains reliability during operation, and provides a stable and usable experience for museum visitors.

The testing process focuses on validating frontend interaction, backend functionality, API communication, intelligent assistant behavior, database consistency, and overall system integration.

6.3.1 Functional Testing

Functional testing is performed to verify that each feature of the system behaves according to the specified requirements.

The following functionalities are included in functional testing:

- Artwork browsing and retrieval.
- Artwork detail display.
- QR-code scanning and artwork access.
- Search and filtering functionality.
- Tour and collection browsing.
- Interactive map navigation.
- User authentication and profile management.
- Saving artworks and personalization features.
- Assistant messaging and smart journey generation.

Each feature is tested using predefined test cases containing expected inputs, expected outputs, and validation conditions.

6.3.2 Frontend Testing

Frontend testing focuses on validating the behavior and usability of the iOS application.

The testing process includes:

- Navigation flow testing.
- UI responsiveness across different device sizes.
- Dark mode compatibility.
- Offline content behavior.
- QR-code scanning interaction.
- Audio playback functionality.
- Error-state and loading-state handling.

User interface consistency and accessibility behavior are also verified during testing.

6.3.3 Backend and API Testing

Backend testing validates REST API behavior, service logic, authentication handling, and database interaction.

The testing process includes:

- Endpoint validation.
- Request and response verification.
- Authentication and authorization testing.
- Database consistency checking.
- Error handling validation.

- Input validation testing.

API endpoints are tested using tools such as Postman to ensure correct communication between the frontend and backend systems.

6.3.4 Database Testing

Database testing ensures that structured museum data is stored and retrieved correctly.

The testing process includes:

- Artwork retrieval validation.
- Relationship consistency testing.
- User data persistence testing.
- Conversation and message storage verification.
- Tour and collection relationship testing.

The integrity of relationships between artworks, artists, rooms, collections, tours, and users is also verified.

6.3.5 Assistant and AI Testing

The intelligent assistant requires additional testing to evaluate response quality and retrieval correctness.

The testing process includes:

- Retrieval relevance testing.
- Context grounding validation.
- Recommendation accuracy testing.
- Invalid-query handling.
- Assistant response consistency.
- Semantic similarity testing.

Special attention is given to reducing hallucinated responses by validating that generated answers remain grounded in museum-specific information retrieved from the vector database.

6.3.6 Performance Testing

Performance testing evaluates the responsiveness and stability of the system under realistic usage conditions.

The testing process includes:

- API response time measurement.
- Concurrent request handling.
- Search performance testing.
- Vector retrieval performance testing.
- Mobile application loading performance.

The objective is to ensure that visitors can retrieve artwork information and assistant responses without significant delay.

6.3.7 Usability Testing

Usability testing evaluates whether visitors can interact with the application efficiently and intuitively.

The testing process focuses on:

- Ease of navigation.
- Readability and accessibility.
- Interaction simplicity.
- Discoverability of features.
- User satisfaction.

Test users may be asked to perform common museum interaction tasks such as searching for an artwork, scanning a QR code, following a tour, or saving an artwork to favorites.

6.3.8 Integration Testing

Integration testing verifies communication between system components.

This includes:

- Frontend-backend communication.
- Backend-database interaction.
- Firebase authentication integration.
- ChromaDB retrieval integration.
- Gemini response generation integration.

The objective is to ensure that all architectural components function correctly as a unified system.

6.3.9 Summary

The testing strategy combines functional, frontend, backend, database, AI, performance, usability, and integration testing to ensure the reliability and quality of the Museum Companion System.

By validating both traditional software functionality and intelligent assistant behavior, the testing process aims to ensure that the system remains stable, usable, accurate, and responsive during real museum usage scenarios.

Chapter 7

Conclusion

The Museum Companion System was proposed as an intelligent and interactive digital platform designed to enhance the museum experience through multimedia interpretation, guided exploration, personalization, and intelligent assistance. The project combines traditional museum information systems with modern mobile technologies, semantic retrieval, and Retrieval-Augmented Generation in order to create a more immersive and accessible cultural experience for visitors.

Throughout the system design process, multiple aspects of software engineering, user experience design, and intelligent system integration were considered. The application was designed not only as a static information viewer, but as a virtual companion capable of supporting visitors during museum exploration through artwork interpretation, QR-code interaction, guided tours, map-based navigation, recommendation mechanisms, and conversational assistance.

From a software architecture perspective, the system adopts a layered client-server design combining an MVVM-based iOS frontend with a modular monolith backend architecture implemented using NestJS. The use of PostgreSQL, ChromaDB, Firebase Authentication, and Gemini 2.5 Flash enables the system to support both structured museum content management and intelligent semantic retrieval functionality.

The relational schema and class design demonstrate a modular and extensible architecture capable of supporting future expansion such as augmented reality, advanced recommendation systems, gamification, multilingual interaction, and enhanced location-aware services.

The project also emphasizes usability and accessibility through careful UI-UX design decisions inspired by museum aesthetics and inclusive interaction principles. Features such as audio narration, transcripts, responsive layouts, dark mode, and intuitive navigation aim to support a broad range of museum visitors with different backgrounds and accessibility needs.

In addition, the integration of Retrieval-Augmented Generation and semantic search represents an important step toward intelligent museum interaction systems. Rather than relying solely on static information retrieval, the assistant system is capable of generating grounded responses based on museum-specific content and contextual retrieval pipelines.

Despite these strengths, the project also encountered several practical limitations during development. One of the most significant limitations of iOS development is the restricted deployment ecosystem imposed by Apple. Deploying applications to physical iOS devices and distributing production-ready builds generally requires an active Apple Developer account, which introduces additional financial and deployment constraints during development and testing.

To mitigate this limitation and ensure broader demonstration capability, a smaller Android prototype version of the application was also developed and submitted. This Android prototype provides a lightweight demonstration of several core features and allows easier deployment and accessibility during evaluation and presentation phases.

Overall, the Museum Companion System demonstrates how modern mobile development, intelligent retrieval systems, and user-centered design principles can be combined to enhance cultural heritage experiences in museum environments. The project provides a strong foundation for future research and development in intelligent museum systems, digital cultural interaction, and AI-assisted educational applications.